Mathematical Foundations for Guidance and Navigation

Part 2. Matrix Fundamentals, Differential Calculation and Jacobians for Navigation

Dr. Weisong Wen, Assistant Professor
Trustworthy AI and Autonomous Systems Laboratory (TAS Lab), The Hong Kong Polytechnic University September 11, 2026

Course Overview: Matrix Fundamentals for Navigation

- **Why study matrix mathematics?**
  - Navigation algorithms rely heavily on matrix operations
  - Sensor data fusion requires matrix manipulations
  - Advanced estimation methods use matrix decompositions
  - Matrix properties determine system stability and accuracy

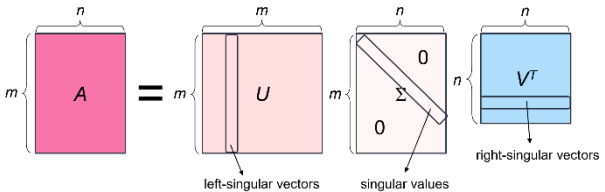


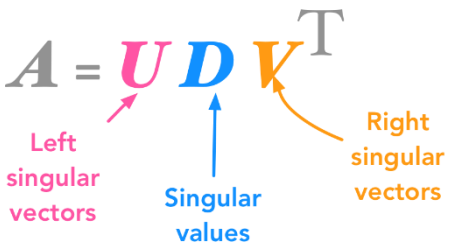
- **Learning objectives:**
  - Understand advanced matrix properties relevant to navigation
  - Master the Singular Value Decomposition (SVD) technique
  - Apply these concepts to navigation problems
  - Recognize when to use different matrix decompositions

- **Key Topics Covered:**
  - Review of fundamental matrix operations and their geometric meaning
  - Eigenvalues, eigenvectors, and matrix decompositions
  - Deep dive into SVD theory and applications
  - Introduction to Jacobian matrices for navigation systems

Lecture Outline: Matrices, SVD and Jacobians

- Review of Basic Matrix Operations
  - Addition, multiplication, transpose, inverse
  - Determinant and special matrices
  - Navigation applications
- Advanced Matrix Properties
  - Eigenvalues and eigenvectors
  - Positive definiteness
  - Matrix condition and stability
- Matrix Decompositions Overview
  - Types of decompositions used in navigation
  - Selecting appropriate decomposition methods
- Singular Value Decomposition (SVD)
  - Theory and intuition
  - Applications to linear systems
  - Solving navigation problems
- Differentiation and Jacobian Matrices
  - Review of differentiation
  - Introduction to Jacobian matrices
  - Navigation applications





Notation and Numerical Precision

One convention throughout the lecture

Scalar: one value, lowercase italic. $a, x_i, \lambda_i, \sigma_i$

Vector: lowercase bold, upright. $\mathbf{x}, \mathbf{v}_i, \mathbf{u}_i$

Matrix: uppercase bold, upright. $\mathbf{A}, \mathbf{P}, \mathbf{J}, \mathbf{U}, \mathbf{\Sigma}, \mathbf{V}$

A vector component is a scalar. Transpose T and descriptive indices are upright. Greek Σ and Λ denote matrices.

Display most results to 2 decimals. Keep 4 decimals for matrix coefficients or small differences when needed. Compute before rounding.

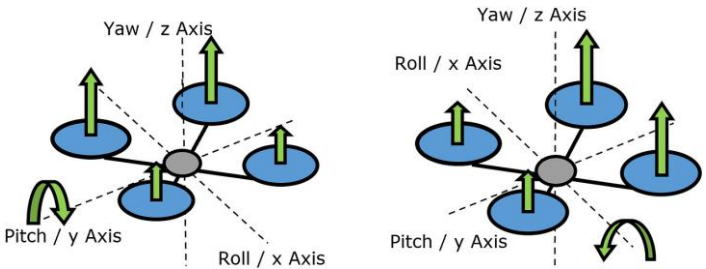


Figure: Matrix computations enable real-time navigation in autonomous systems

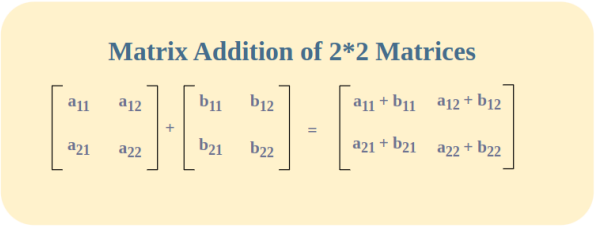
Basic Matrix Operations Review - I: Addition & Subtraction

Matrix Addition & Subtraction
For matrices of same dimensions:

$$\mathbf{A} + \mathbf{B} = [a_{ij} + b_{ij}]$$
$$\mathbf{A} - \mathbf{B} = [a_{ij} - b_{ij}]$$

Example:

$$\begin{bmatrix} 1 & 2 \\ 3 & 4 \end{bmatrix} + \begin{bmatrix} 5 & 6 \\ 7 & 8 \end{bmatrix} = \begin{bmatrix} 6 & 8 \\ 10 & 12 \end{bmatrix}$$



Properties:

- Commutative: $\mathbf{A} + \mathbf{B} = \mathbf{B} + \mathbf{A}$
- Associative: $(\mathbf{A} + \mathbf{B}) + \mathbf{C} = \mathbf{A} + (\mathbf{B} + \mathbf{C})$
- Identity: $\mathbf{A} + 0 = \mathbf{A}$
- Additive inverse: $\mathbf{A} + (-\mathbf{A}) = 0$

Navigation application:

- Combining multiple coordinate offsets
- Averaging multiple position estimates
- Combining error terms in navigation filters
- Adding displacement vectors in path planning

Basic Matrix Operations Review - II: Multiplication

Matrix Multiplication

For $\mathbf{A}_{m\times n}$ and $\mathbf{B}_{n\times p}$:

$$(\mathbf{AB})_{\mathrm{ij}} = \sum_{k=1}^{n} a_{\mathrm{ik}}\, b_{\mathrm{kj}}$$

Example:

$$\begin{bmatrix}1 & 2\\ 3 & 4\end{bmatrix}\begin{bmatrix}5 & 6\\ 7 & 8\end{bmatrix}$$

$$= \begin{bmatrix}1\times 5+2\times 7 & 1\times 6+2\times 8\\ 3\times 5+4\times 7 & 3\times 6+4\times 8\end{bmatrix}$$

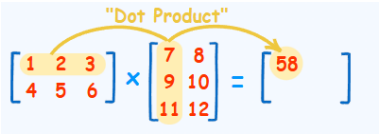
$$= \begin{bmatrix}19 & 22\\ 43 & 50\end{bmatrix}$$

Properties:

- Not commutative: $\mathbf{AB} \neq \mathbf{BA}$ (generally)
- Associative: $(\mathbf{AB})\mathbf{C} = \mathbf{A}(\mathbf{BC})$
- Distributive: $\mathbf{A}(\mathbf{B}+\mathbf{C}) = \mathbf{AB}+\mathbf{AC}$
- Identity: $\mathbf{AI} = \mathbf{IA} = \mathbf{A}$

Navigation application:

- Applying coordinate transformations
- Converting vectors between reference frames
- Sequential rotations in 3D space
- State transition in navigation algorithms



Basic Matrix Operations Review - III: Transpose

Matrix Transpose
Flips a matrix over its diagonal:

$$\left(\mathbf{A}^{\mathrm{T}}\right)_{\mathrm{ij}} = a_{\mathrm{ji}}$$

Example:

$$\begin{bmatrix} 1 & 2 & 3 \\ 4 & 5 & 6 \end{bmatrix}^{\mathrm{T}} = \begin{bmatrix} 1 & 4 \\ 2 & 5 \\ 3 & 6 \end{bmatrix}$$

Properties:

$$\left(\mathbf{A}^{\mathrm{T}}\right)^{\mathrm{T}} = \mathbf{A}$$
$$(\mathbf{A} + \mathbf{B})^{\mathrm{T}} = \mathbf{A}^{\mathrm{T}} + \mathbf{B}^{\mathrm{T}}$$
$$(\mathbf{AB})^{\mathrm{T}} = \mathbf{B}^{\mathrm{T}}\mathbf{A}^{\mathrm{T}}$$

Special cases:

- **Symmetric matrix**: $\mathbf{A}^{\mathrm{T}} = \mathbf{A}$
  — Example: Covariance matrices
  — $\mathbf{P} = \begin{bmatrix} \sigma_{\mathrm{x}}^2 & \sigma_{\mathrm{xy}} \\ \sigma_{\mathrm{xy}} & \sigma_{\mathrm{y}}^2 \end{bmatrix}$
- **Skew-symmetric matrix**: $\mathbf{A}^{\mathrm{T}} = -\mathbf{A}$
  — Example: Cross-product operator

$$[\vec{\boldsymbol{\omega}}]_{\times} = \begin{bmatrix} 0 & -\omega_{\mathrm{z}} & \omega_{\mathrm{y}} \\ \omega_{\mathrm{z}} & 0 & -\omega_{\mathrm{x}} \\ -\omega_{\mathrm{y}} & \omega_{\mathrm{x}} & 0 \end{bmatrix}$$

Navigation application:

- Converting between inertial and body coordinates
- Efficient attitude representation
- Error propagation calculations

Basic Matrix Operations Review - IV: Inverse

Matrix Inverse

For square matrix $\mathbf{A}$, if it exists:

$$\mathbf{A}\mathbf{A}^{-1} = \mathbf{A}^{-1}\mathbf{A} = \mathbf{I}$$

Properties:

$$(\mathbf{A}^{-1})^{-1} = \mathbf{A}$$
$$\left(\mathbf{A}^{\mathrm{T}}\right)^{-1} = (\mathbf{A}^{-1})^{\mathrm{T}}$$
$$(\mathbf{AB})^{-1} = \mathbf{B}^{-1}\mathbf{A}^{-1}$$

2×2 Matrix Inverse Formula:

$$\begin{bmatrix} a & b \\ c & d \end{bmatrix}^{-1} = \frac{1}{ad - bc}\begin{bmatrix} d & -b \\ -c & a \end{bmatrix}$$

Example Calculation:

Find the inverse of $\mathbf{A} = \begin{bmatrix} 2 & 1 \\ 1 & 3 \end{bmatrix}$

Step 1: Calculate determinant

$$\det(\mathbf{A}) = 2 \times 3 - 1 \times 1 = 5$$

Step 2: Apply formula

$$\mathbf{A}^{-1} = \frac{1}{5}\begin{bmatrix} 3 & -1 \\ -1 & 2 \end{bmatrix} = \begin{bmatrix} 0.6 & -0.2 \\ -0.2 & 0.4 \end{bmatrix}$$

Navigation application:

- Solving systems of linear equations
- Reversing coordinate transformations
- Computing least-squares solutions in positioning

Matrix Calculations and Properties - V: Determinant

Determinant
For a 2×2 matrix:

$$\det \begin{bmatrix} a & b \\ c & d \end{bmatrix} = ad - bc$$

Example:

$$\det \begin{bmatrix} 4 & 2 \\ 1 & 3 \end{bmatrix} = 4 \times 3 - 2 \times 1 = 10$$

Properties:

$$\det(\mathbf{AB}) = \det(\mathbf{A})\det(\mathbf{B})$$

$$det\left(\mathbf{A}^{\mathrm{T}}\right) = \det(\mathbf{A})$$

$$\det(\mathbf{A}^{-1}) = \frac{1}{\det(\mathbf{A})}$$

Physical meaning in 3D transformations:

- $|\det(\mathbf{A})|$: Volume scaling factor
- $\det(\mathbf{A}) = 0$: Non-invertible (singular)

Examples in navigation:

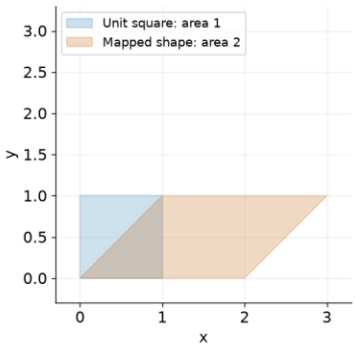
- Rotation matrices: $\det(\mathbf{F}) = 1$
- Reflection matrices: $\det(\mathbf{F}) = -1$
- Used to check validity of transformation matrices

Navigation application:

- Testing if a matrix is singular before inversion

Determinant Example: Area and Invertibility

A unit square becomes a parallelogram. Follow its vertices to see the effect of the matrix.



The image has base 2 and height 1, so its area is 2.

The determinant is 2. The transformation preserves orientation.

A zero determinant would flatten the square into a line or point.

$$\mathbf{A} = \begin{bmatrix} 2 & 1 \\ 0 & 1 \end{bmatrix}, \det(\mathbf{A}) = 2 \times 1 - 0 \times 1 = 2$$

Special Matrices in Navigation - VI

Orthogonal Matrices

A square matrix $\mathbf{Q}$ is orthogonal if:

$$\mathbf{Q}^{\mathrm{T}}\mathbf{Q} = \mathbf{Q}\mathbf{Q}^{\mathrm{T}} = \mathbf{I}$$

Key properties:

- $\mathbf{Q}^{-1} = \mathbf{Q}^{\mathrm{T}}$
- $\det(\mathbf{Q}) = \pm 1$
- Preserves lengths and angles
- Columns form orthonormal basis

Example: 2D rotation matrix

$$\mathbf{R}(\theta) = \begin{bmatrix} cos\,\theta & -sin\,\theta \\ sin\,\theta & cos\,\theta \end{bmatrix}$$

Navigation applications:

- Attitude representation
- Direction Cosine Matrices (DCM)
- Coordinate frame transformations

Symmetric Matrices

A matrix $\mathbf{S}$ is symmetric if:

$$\mathbf{S}^{\mathrm{T}} = \mathbf{S} \qquad (28)$$

Key properties:

- All eigenvalues are real
- Eigenvectors from different eigenvalues are orthogonal
- Can be diagonalized by orthogonal matrix

Example: Covariance matrix

$$\mathbf{P} = \begin{bmatrix} \sigma_{\mathrm{x}}^2 & \sigma_{\mathrm{xy}} \\ \sigma_{\mathrm{xy}} & \sigma_{\mathrm{y}}^2 \end{bmatrix}$$

Navigation applications:

- Position error representation
- Inertia tensors for spacecraft
- Kalman filter covariance matrices

Advanced Matrix Properties - I



Review from Coordinate Transformations

- Rotation matrices are orthogonal: $\mathbf{R}\mathbf{R}^{\mathrm{T}} = \mathbf{I}$
- Their determinant equals 1: $\det(\mathbf{R}) = 1$
- Their inverse equals their transpose: $\mathbf{R}^{-1} = \mathbf{R}^{\mathrm{T}}$

Eigenvalues and Eigenvectors

- Definition: For a square matrix $\mathbf{A}$, a non-zero vector $\mathbf{v}$ is an eigenvector with eigenvalue λ if:

$$\mathbf{A}\mathbf{v} = \lambda\mathbf{v} \tag{30}$$

- Solve by finding roots of characteristic equation:

$$\det(\mathbf{A} - \lambda\mathbf{I}) = 0 \tag{31}$$

- Physical meaning: Directions that are only scaled (not rotated) by the transformation

Key Properties

A real n × n matrix has n complex eigenvalues, counting multiplicity.
Nonreal eigenvalues occur in conjugate pairs.
Symmetric matrices have real eigenvalues.
Rotation eigenvalues have magnitude one.

Applications in Navigation

- System stability analysis
- Principal component analysis for sensor data
- Modal analysis of dynamic systems
- Error ellipsoid visualization
- Determining principal axes of inertia
- Optimizing computational efficiency

Eigenvalues of a Symmetric Matrix

Step 1: Characteristic equation

$$\mathbf{A} = \begin{bmatrix} 3 & 1 \\ 1 & 3 \end{bmatrix}$$

$$\det(\mathbf{A} - \lambda \mathbf{I}) = (3 - \lambda)^2 - 1 = 0$$

$$(3 - \lambda)^2 = 1$$

$$3 - \lambda = \pm 1$$

Step 2: Eigenvalues

$$\lambda_1 = 2, \qquad \lambda_2 = 4$$

Step 3: Eigenvector for the value 2

$$(\mathbf{A} - 2\mathbf{I})\mathbf{v}_1 = \begin{bmatrix} 1 & 1 \\ 1 & 1 \end{bmatrix} \mathbf{v}_1 = \begin{bmatrix} 0 \\ 0 \end{bmatrix}$$

$$v_{11} + v_{12} = 0$$

$$\mathbf{v}_1 = \begin{bmatrix} 1 \\ -1 \end{bmatrix}$$

Step 4: Eigenvector for the value 4

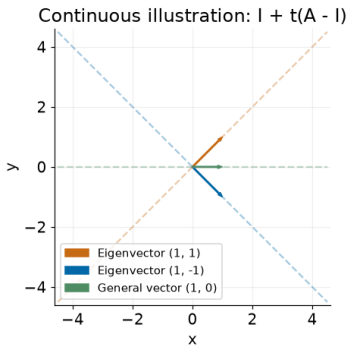
$$(\mathbf{A} - 4\mathbf{I})\mathbf{v}_2 = \begin{bmatrix} -1 & 1 \\ 1 & -1 \end{bmatrix} \mathbf{v}_2 = \begin{bmatrix} 0 \\ 0 \end{bmatrix}$$

$$\mathbf{v}_2 = \begin{bmatrix} 1 \\ 1 \end{bmatrix}$$

Check: each product equals its eigenvalue times the vector.

Eigenvectors: Direction Versus Length

At the end of the animation, the matrix is the same symmetric matrix used in the example.



The orange vector ends at (4, 4): its eigenvalue is 4.

The blue vector ends at (2, -2): its eigenvalue is 2.

The green vector ends at (3, 1). Its direction changes, so it is not an eigenvector.

$$\mathbf{A} = \begin{bmatrix} 3 & 1 \\ 1 & 3 \end{bmatrix}$$

Eigenvalue Calculation: Example 2

Example 2: Rotation Matrix
Consider a 2D rotation matrix for 90° rotation:

$$\mathbf{R} = \begin{bmatrix} 0 & -1 \\ 1 & 0 \end{bmatrix}$$

Step 1: Form the characteristic equation

$$\det(\mathbf{R} - \lambda \mathbf{I}) = 0$$
$$\det \begin{bmatrix} -\lambda & -1 \\ 1 & -\lambda \end{bmatrix} = 0$$
$$\lambda^2 + 1 = 0$$

Step 2: Solve for eigenvalues

$$\lambda^2 = -1$$
$$\lambda = \pm \mathrm{i}$$

The eigenvalues are $\lambda_1 = \mathrm{i}\ and\ \lambda_2 = -\mathrm{i}$

Step 3: Find eigenvectors for $\lambda_1 = \mathrm{i}$

$$(\mathbf{R} - \mathrm{i}\mathbf{I})\mathbf{v}_1 = 0$$
$$\begin{bmatrix} -\mathrm{i} & -1 \\ 1 & -\mathrm{i} \end{bmatrix} \begin{bmatrix} v_{11} \\ v_{12} \end{bmatrix} = \begin{bmatrix} 0 \\ 0 \end{bmatrix}$$

This gives us $-\mathrm{i}v_{11} - v_{12} = 0, v_{12} = -\mathrm{i}v_{11}$

Taking $v_{11} = 1$, we get $\mathbf{v}_1 = \begin{bmatrix} 1 \\ -\mathrm{i} \end{bmatrix}$

Step 4: Find eigenvectors for $\lambda_2 = -\mathrm{i}$

$$(\mathbf{R} + \mathrm{i}\mathbf{I})\mathbf{v}_2 = 0$$
$$\begin{bmatrix} \mathrm{i} & -1 \\ 1 & \mathrm{i} \end{bmatrix} \begin{bmatrix} v_{21} \\ v_{22} \end{bmatrix} = \begin{bmatrix} 0 \\ 0 \end{bmatrix}$$

This gives us $\mathrm{i}v_{21} - v_{22} = 0,\ so\ v_{22} = \mathrm{i}v_{21}$

Taking $v_{21} = 1$, we get $\mathbf{v}_2 = \begin{bmatrix} 1 \\ \mathrm{i} \end{bmatrix}$

Physical meaning: A 90° rotation has no real eigenvectors!

Covariance Eigenvalues: Worked Example

Given: horizontal GNSS covariance

$$\mathbf{P} = \begin{bmatrix} 25 & 15 \\ 15 & 16 \end{bmatrix} \mathrm{m}^2$$

Step 1: Expand the determinant

$$\det(\mathbf{P} - \lambda \mathbf{I}) = 0$$

$$(25 - \lambda)(16 - \lambda) - 15^2 = 0$$

$$\lambda^2 - 41\lambda + 175 = 0$$

The trace is 41 and the determinant is 175. These give independent checks on the two roots.

Step 2: Solve the quadratic

$$\lambda = \frac{41 \pm \sqrt{41^2 - 4 \times 175}}{2}$$

$$\lambda = \frac{41 \pm \sqrt{981}}{2}$$

$$\lambda_1 \approx 36.16\,\mathrm{m}^2$$

$$\lambda_2 \approx 4.84\,\mathrm{m}^2$$

Eigenvalues are variances. Their square roots give standard deviations along the principal axes.

From Covariance to an Error Ellipse

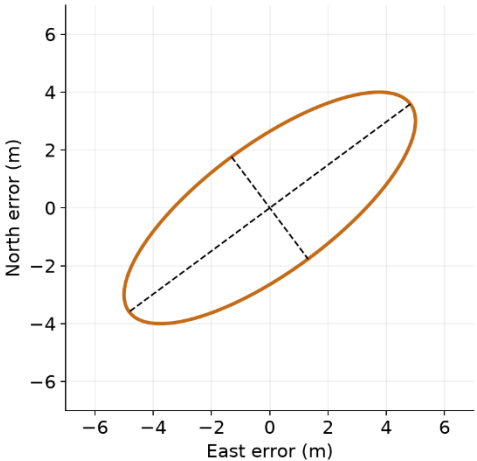
Principal directions and uncertainty

$\sqrt{\lambda_1} \approx 6.01\,\mathrm{m}$

$\sqrt{\lambda_2} \approx 2.20\,\mathrm{m}$

Eigenvectors set the ellipse directions. Larger eigenvalues mean greater uncertainty along those directions.

This unit Mahalanobis contour contains about 39.35% of a 2D Gaussian. A 95% contour uses an axis multiplier of about 2.45.



Eigenvalue Calculation for Scaling Matrix (Part 1)

Matrix from Visual Example

$$\mathbf{A} = \begin{bmatrix} 2 & 0 \\ 0 & 0.5 \end{bmatrix}$$

Step 1: Form the characteristic equation

$$\det(\mathbf{A} - \lambda \mathbf{I}) = 0$$

$$\det \begin{bmatrix} 2-\lambda & 0 \\ 0 & 0.5-\lambda \end{bmatrix} = 0$$

For a diagonal matrix, the determinant is the product of the diagonal elements:

$$(2-\lambda)(0.5-\lambda) = 0$$

$$\lambda^2 - 2.5\lambda + 1 = 0$$

Step 2: Solve for eigenvalues
The equation factorizes directly:

$$(2-\lambda)(0.5-\lambda) = 0$$

$$\lambda = 2 \text{ or } \lambda = 0.5$$

Eigenvalue Calculation for Scaling Matrix (Part 2A)

Step 3: Find eigenvector for $\lambda_1 = 2$

$$(\mathbf{A} - 2\mathbf{I})\mathbf{v}_1 = 0$$

$$\begin{bmatrix} 0 & 0 \\ 0 & -1.5 \end{bmatrix} \begin{bmatrix} v_{11} \\ v_{12} \end{bmatrix} = \begin{bmatrix} 0 \\ 0 \end{bmatrix}$$

This simplifies to:

$$0 \cdot v_{11} + 0 \cdot v_{12} = 0$$

$$0 \cdot v_{11} + (-1.5) \cdot v_{12} = 0$$

The second equation gives us: $-1.5\mathbf{v}_{12} = 0$, which means $\mathbf{v}_{12} = 0$

The equation is satisfied for any value of $\mathbf{v}_{11}$

Taking $\mathbf{v}_{11} = 1$ to normalize:

$$\mathbf{v}_1 = \begin{bmatrix} 1 \\ 0 \end{bmatrix}$$

Verification for $\lambda_1 = 2$:

$$\mathbf{A}\mathbf{v}_1 = \begin{bmatrix} 2 & 0 \\ 0 & 0.5 \end{bmatrix} \begin{bmatrix} 1 \\ 0 \end{bmatrix} = \begin{bmatrix} 2 \\ 0 \end{bmatrix} = 2 \begin{bmatrix} 1 \\ 0 \end{bmatrix} = 2\mathbf{v}_1$$

Eigenvalue Calculation for Scaling Matrix (Part 2B)

Step 4: Find eigenvector for $\lambda_2 = 0.5$

$$(\mathbf{A} - 0.5\mathbf{I})\mathbf{v}_2 = 0$$

$$\begin{bmatrix} 1.5 & 0 \\ 0 & 0 \end{bmatrix} \begin{bmatrix} v_{21} \\ v_{22} \end{bmatrix} = \begin{bmatrix} 0 \\ 0 \end{bmatrix}$$

This simplifies to:

$$1.5 \cdot v_{21} + 0 \cdot v_{22} = 0$$
$$0 \cdot v_{21} + 0 \cdot v_{22} = 0$$

The first equation gives us: $1.5v_{21} = 0$, which means $\mathbf{v}_{21} = 0$

The second equation is satisfied for any value of $\mathbf{v}_{22}$

Taking $\mathbf{v}_{22} = 1$ to normalize:

$$\mathbf{v}_2 = \begin{bmatrix} 0 \\ 1 \end{bmatrix}$$

Verification for $\lambda_2 = 0.5$:

$$\mathbf{A}\mathbf{v}_2 = \begin{bmatrix} 2 & 0 \\ 0 & 0.5 \end{bmatrix} \begin{bmatrix} 0 \\ 1 \end{bmatrix} = \begin{bmatrix} 0 \\ 0.5 \end{bmatrix} = 0.5 \begin{bmatrix} 0 \\ 1 \end{bmatrix} = 0.5\mathbf{v}_2$$

Eigenvalue Calculation: Summary and Key Insights

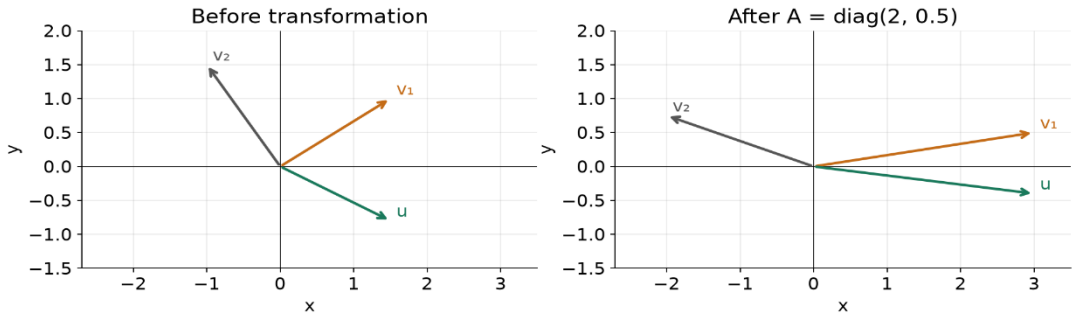
Results Summary:

- Eigenvalues: $\lambda_1 = 2 \; and \; \lambda_2 = 0.5$
- Eigenvectors: $\mathbf{v}_1 = \begin{bmatrix} 1 \\ 0 \end{bmatrix} \; and \; \mathbf{v}_2 = \begin{bmatrix} 0 \\ 1 \end{bmatrix}$

For this diagonal matrix, the coordinate axes are eigenvector directions.
The diagonal entries are its eigenvalues.
The x component is stretched by 2, and the y component is compressed by 0.5.
Every vector can be expanded in this eigenvector basis.
A general matrix needs a complete eigenvector basis for this form of decomposition.

Eigenvectors and General Vectors under Scaling

The coordinate axes are eigenvector directions of this diagonal matrix. The three oblique vectors below are general vectors.



A doubles x and halves y. General vectors change direction. A real eigenvector stays on its own line, reversing direction if its eigenvalue is negative.

Connecting Calculations to Visualization (Part 1)

Vectors from Our Visual Example
In our visualization, we used:

$$\mathbf{v}_1 = \begin{bmatrix} 1.5 \\ 1 \end{bmatrix}$$

$$\mathbf{v}_2 = \begin{bmatrix} -1 \\ 1.5 \end{bmatrix}$$

$$\mathbf{u} = \begin{bmatrix} 1.5 \\ -0.8 \end{bmatrix}$$

Decomposition into Eigenvectors
Any vector can be written as a linear combination of eigenvectors:

$$\mathbf{v} = c_1\mathbf{e}_1 + c_2\mathbf{e}_2$$

Where $\mathbf{e}_1 = \begin{bmatrix} 1 \\ 0 \end{bmatrix}$ and $\mathbf{e}_2 = \begin{bmatrix} 0 \\ 1 \end{bmatrix}$ are our calculated eigenvectors.

Expressing Our Vectors

$$\mathbf{v}_1 = 1.5\mathbf{e}_1 + 1\mathbf{e}_2$$
$$\mathbf{v}_2 = -1\mathbf{e}_1 + 1.5\mathbf{e}_2$$
$$\mathbf{u} = 1.5\mathbf{e}_1 - 0.8\mathbf{e}_2$$

Transformation Property
For any vector $\mathbf{v} = c_1\mathbf{e}_1 + c_2\mathbf{e}_2$:

$$\begin{aligned} \mathbf{A}\mathbf{v} &= \mathbf{A}(c_1\mathbf{e}_1 + c_2\mathbf{e}_2) \\ &= c_1\mathbf{A}\mathbf{e}_1 + c_2\mathbf{A}\mathbf{e}_2 \\ &= c_1\lambda_1\mathbf{e}_1 + c_2\lambda_2\mathbf{e}_2 \\ &= 2c_1\mathbf{e}_1 + 0.5c_2\mathbf{e}_2 \end{aligned}$$

Connecting Calculations to Visualization (Part 2)

Transforming Vector $\mathbf{v}_1$

$$\mathbf{A}\mathbf{v}_1 = \begin{bmatrix} 2 & 0 \\ 0 & 0.5 \end{bmatrix} \begin{bmatrix} 1.5 \\ 1 \end{bmatrix}$$
$$= \begin{bmatrix} 2 \times 1.5 \\ 0.5 \times 1 \end{bmatrix}$$
$$= \begin{bmatrix} 3 \\ 0.5 \end{bmatrix}$$

Alternatively, using eigenvector decomposition:

$$\mathbf{A}\mathbf{v}_1 = \mathbf{A}(1.5\mathbf{e}_1 + 1\mathbf{e}_2)$$
$$= 1.5(2\mathbf{e}_1) + 1(0.5\mathbf{e}_2)$$
$$= 3\mathbf{e}_1 + 0.5\mathbf{e}_2$$
$$= \begin{bmatrix} 3 \\ 0.5 \end{bmatrix}$$

Transforming Vector $\mathbf{v}_2$

$$\mathbf{A}\mathbf{v}_2 = \begin{bmatrix} 2 & 0 \\ 0 & 0.5 \end{bmatrix} \begin{bmatrix} -1 \\ 1.5 \end{bmatrix}$$
$$= \begin{bmatrix} 2 \times (-1) \\ 0.5 \times 1.5 \end{bmatrix}$$
$$= \begin{bmatrix} -2 \\ 0.75 \end{bmatrix}$$

Transforming Vector u

$$\mathbf{A}\mathbf{u} = \begin{bmatrix} 2 & 0 \\ 0 & 0.5 \end{bmatrix} \begin{bmatrix} 1.5 \\ -0.8 \end{bmatrix}$$
$$= \begin{bmatrix} 2 \times 1.5 \\ 0.5 \times (-0.8) \end{bmatrix}$$
$$= \begin{bmatrix} 3 \\ -0.4 \end{bmatrix}$$

Class Exercise: Covariance Eigenvectors

Horizontal position uncertainty in ENU

$$\mathbf{P} = \begin{bmatrix} 9 & 6 \\ 6 & 16 \end{bmatrix} \mathrm{m}^2$$

East and North errors form this 2D covariance. Treat the vertical component separately.

1. Calculate the eigenvalues and normalized eigenvectors.
2. Verify the eigenvalue equations and orthogonality.
3. Explain the principal directions in navigation terms.
4. Construct the contour with semi-axes $2\sqrt{\lambda_1}$ and $2\sqrt{\lambda_2}$.
5. Discuss how the geometry affects position uncertainty.

For a 2D Gaussian, this “2σ ellipse” contains about 86.47%, rather than the 1D interval probability of about 95.45%.

Covariance Exercise: Calculation of Principal Axes

Given horizontal position covariance

$$\mathbf{P} = \begin{bmatrix} 9 & 6 \\ 6 & 16 \end{bmatrix}$$

Step 1: Characteristic equation

$$\det(\mathbf{P} - \lambda \mathbf{I}) = (9 - \lambda)(16 - \lambda) - 36$$

$$\lambda^2 - 25\lambda + 108 = 0$$

$$\lambda_1 \approx 19.45, \qquad \lambda_2 \approx 5.55$$

Step 2: Unit eigenvectors

$$(9 - \lambda) v_E + 6 v_N = 0$$

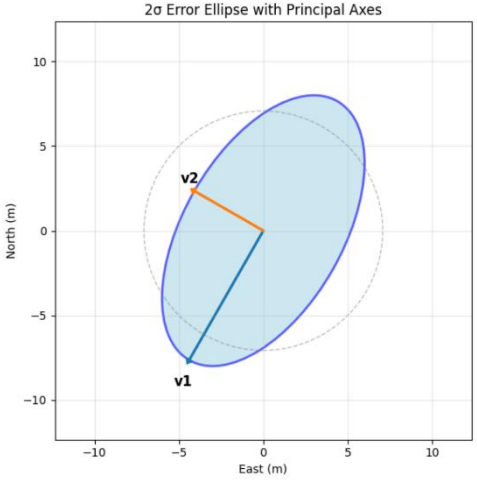
For the larger eigenvalue, the North-to-East component ratio is about 1.74. Normalize to unit length.

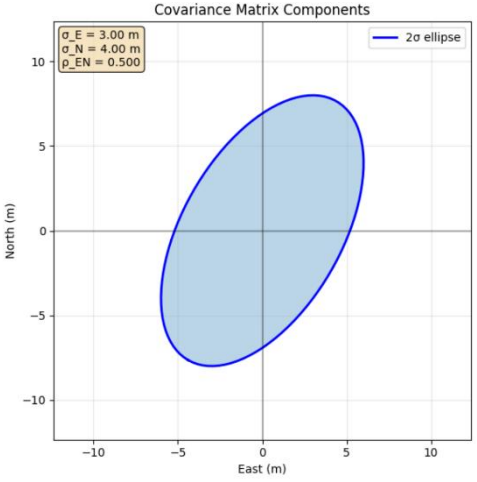
$$\mathbf{v}_1 \approx \begin{bmatrix} 0.4981 \\ 0.8671 \end{bmatrix}, \mathbf{v}_2 \approx \begin{bmatrix} -0.8671 \\ 0.4981 \end{bmatrix}$$

Either sign of an eigenvector is valid. The two directions must be perpendicular.

Covariance Exercise: Interpretation of the Original Figure

Eigenvalues are variances: 2σ semi-axes are 8.82 m and 4.71 m. Correlation is 6/(3 × 4) = 0.50.

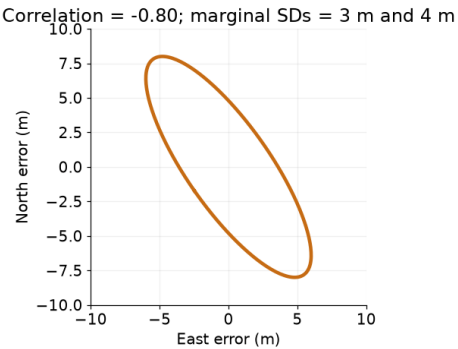




For a 2D Gaussian, this contour contains 86.47%, not the 1D value 95.45%.

Covariance Animation: What Correlation Changes

Keep the East and North standard deviations fixed. Change only their correlation.



The East and North variances stay at 9 and 16 square metres.

Positive correlation tilts the long axis up and right. Negative correlation reverses the tilt.

At zero correlation, the principal axes align with East and North.

$$\mathbf{P} = \begin{bmatrix} 9 & 12\rho \\ 12\rho & 16 \end{bmatrix}$$

Eigenvalues and Eigenvectors: Why They Matter in Navigation



Theoretical Importance

- Fundamental property of linear transformations
- Simplify complex matrix operations
- Enable matrix diagonalization
- Provide insight into system stability
- Characterize key directions in data

Mathematical Applications

- Principal Component Analysis (PCA)
- Solving coupled differential equations
- Optimizing computational efficiency
- Analyzing dynamic systems

Navigation Applications

- **Error Ellipses:** Semi-major and semi-minor axes of position error ellipses in GNSS
- **Inertial Navigation:** Principal axes of inertia in vehicle dynamics
- **Sensor Fusion:** Optimally combining data from multiple sensors
- **Attitude Determination:** Finding optimal rotation from measurements
- **Kalman Filtering:** Understanding error propagation in state estimation

Key Takeaway: Eigenvalues and eigenvectors provide critical insight into transformations and error characteristics, helping us design robust navigation systems.

Positive Definite Matrices

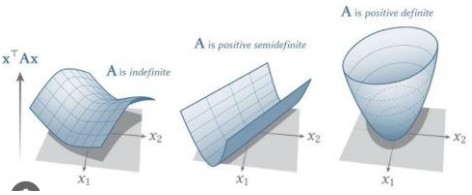
- A symmetric matrix **P** is positive definite if:

$$\mathbf{x}^{\mathrm{T}}\mathbf{P}\mathbf{x} > 0 \quad \text{for all non-zero vectors } \mathbf{x}$$

- Equivalent conditions:
  - All eigenvalues are positive
  - All leading principal minors are positive
  - $\mathbf{P} = \mathbf{L}\mathbf{L}^{\mathrm{T}}$ for some non-singular $\mathbf{L}$ (Cholesky decomposition)
- A matrix is positive semi-definite if:

$$\mathbf{x}^{\mathrm{T}}\mathbf{P}\mathbf{x} \geq 0 \text{ for all vectors } \mathbf{x} \qquad (110)$$

Navigation applications
Covariance matrices are positive semidefinite. They are positive definite when all nonzero directions have positive variance.
An invertible covariance defines an information matrix.
Positive definite weights give a strict quadratic penalty.



Condition Number and Sensitivity

A square, nonsingular linear system

$$\mathbf{Ax} = \mathbf{b}$$

$$\kappa_2(\mathbf{A}) = \|\mathbf{A}\|_2 \left\|\mathbf{A}^{-1}\right\|_2 = \frac{\sigma_1}{\sigma_n}$$

The singular-value ratio applies to the 2-norm. For a singular square matrix, the condition number is infinite.

$$\frac{\|\Delta x\|_2}{\|x\|_2} \leq \kappa_2(\mathbf{A}) \frac{\|\Delta b\|_2}{\|b\|_2}$$

What the bound tells us

Hold the matrix fixed and perturb the data vector. The condition number bounds the possible relative error amplification.

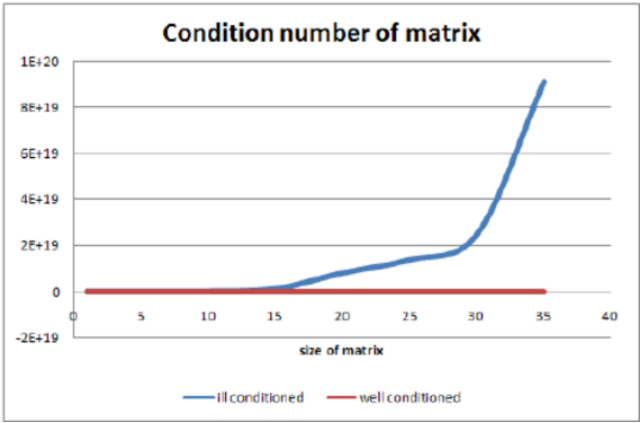
A large value indicates sensitivity, even if the numerical solver is stable. It does not give the exact error for every input.

Navigation interpretation

Similar satellite directions leave some position components weakly observed. Scaling the state variables and inspecting the singular values helps diagnose this.

Condition Number: Reading the Original Plot

The original illustration compares a well-conditioned family with an ill-conditioned family.



Near 1: little worst-case relative amplification.

Large value: some perturbations may be strongly amplified.

The plotted matrices are unspecified. Matrix size alone does not determine conditioning: an identity matrix has condition number 1 at every size.

Condition Number: How Large Is Large?



Judge the condition number against input accuracy and the error you can tolerate.

$$\kappa_2(\mathbf{A}) = \frac{\sigma_1}{\sigma_n}$$

Example: relative data uncertainty = 0.01%

κ = 1: bound on relative solution change = 0.01%
κ = 100: bound = 1%
κ = 10^4: bound = 100%

These are worst-case bounds for fixed, nonsingular A and perturbations in b. Actual errors depend on their direction.

There is no universal cutoff. A singular square matrix has infinite condition number.

Condition Number Example: A Sensitive Direction

Step 1: Set up two equations

$$\mathbf{A} = \begin{bmatrix} 1 & 0 \\ 0 & 0.01 \end{bmatrix}, \mathbf{b} = \begin{bmatrix} 1 \\ 0 \end{bmatrix}$$

$$x_1 = 1, \qquad 0.01x_2 = 0$$

$$\mathbf{x} = \begin{bmatrix} 1 \\ 0 \end{bmatrix}$$

Step 2: Compute conditioning

$$\sigma_1 = 1, \qquad \sigma_2 = 0.01$$

$$\kappa_2(\mathbf{A}) = \frac{1}{0.01} = 100$$

The second equation is weakly sensitive to x_2. Dividing its measurement error by 0.01 amplifies that error by 100.

All quantities use consistent, nondimensional units in this teaching example.

Condition Number Example: Perturb and Recompute

Step 3: Add a small data error

$$\Delta\mathbf{b} = \begin{bmatrix} 0 \\ 0.0001 \end{bmatrix}$$

$$0.01\Delta x_2 = 0.0001$$

$$\Delta\mathbf{x} = \begin{bmatrix} 0 \\ 0.01 \end{bmatrix}$$

Step 4: Compare relative changes

$$\frac{\|\Delta\mathbf{b}\|}{\|\mathbf{b}\|} = 0.0001 = 0.01\%$$

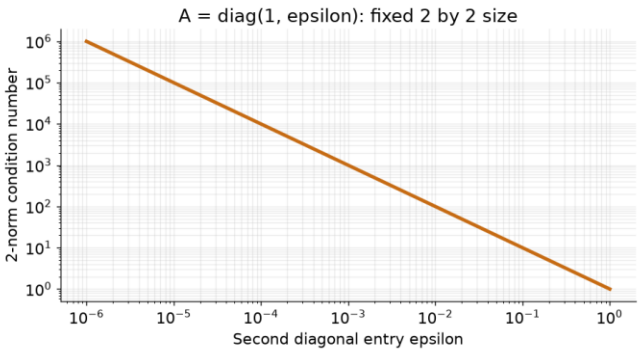
$$\frac{\|\Delta\mathbf{x}\|}{\|\mathbf{x}\|} = 0.01 = 1\%$$

The relative change grows by a factor of 100, reaching the condition-number bound in this direction.

For comparison, A = I gives Δx = Δb and amplification 1. This is a property of the problem, even with exact arithmetic.

Conditioning Experiment: Same Size, Different Sensitivity

Every matrix in this plot is 2 by 2. Only its second diagonal entry changes.



For epsilon = 1, the condition number is 1.

For epsilon = 0.01, it is 100. For 0.000001, it is one million.

Small epsilon weakens the second measurement direction. At zero, the matrix is singular.

$$\kappa_2(\mathbf{A}) = \frac{1}{\varepsilon}$$

Matrix Decompositions Overview



Eigendecomposition

$$\mathbf{A} = \mathbf{Q}\boldsymbol{\Lambda}\mathbf{Q}^{-1}$$

Requires a complete eigenvector basis. Reveals invariant directions and scaling.

Singular value decomposition

$$\mathbf{A} = \mathbf{U}\boldsymbol{\Sigma}\mathbf{V}^{\mathrm{T}}$$

Exists for any real rectangular matrix. Reveals rank and supports stable least-squares calculations.

QR decomposition

$$\mathbf{A} = \mathbf{QR}$$

Orthogonal columns and an upper-triangular factor. Useful for least squares.

Cholesky decomposition

$$\mathbf{A} = \mathbf{L}\mathbf{L}^{\mathrm{T}}$$

For symmetric positive definite matrices. The triangular factor is efficient to solve with.

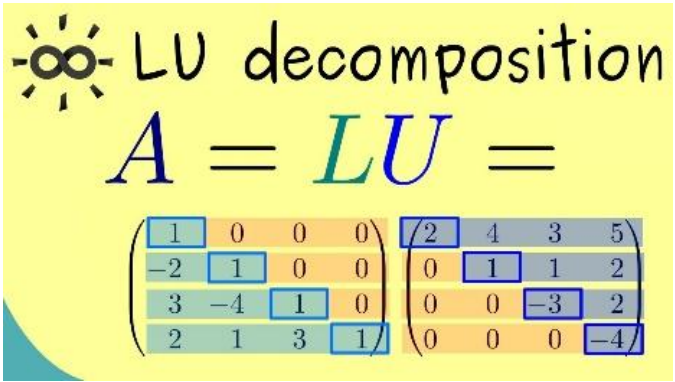
Pivoted LU decomposition

$$\mathbf{PA} = \mathbf{LU}$$

P permutes rows. Useful for square systems.

LU Decomposition: The Original Visual Explanation

Triangular factors organize Gaussian elimination. The diagonal of L is one in this convention.



L stores elimination multipliers below its diagonal.

U is upper triangular, so solve from the last equation upward.

The picture illustrates A = LU. With row pivoting, write PA = LU.

LU Worked Example: Elimination and Substitution

Step 1: Eliminate below the pivot

$$\mathbf{A} = \begin{bmatrix} 2 & 1 \\ 4 & 3 \end{bmatrix}, \mathbf{b} = \begin{bmatrix} 1 \\ 3 \end{bmatrix}$$

Subtract twice row 1 from row 2. The multiplier is 4/2 = 2.

$$\mathbf{L} = \begin{bmatrix} 1 & 0 \\ 2 & 1 \end{bmatrix}, \mathbf{U} = \begin{bmatrix} 2 & 1 \\ 0 & 1 \end{bmatrix}$$

Check: multiplying L and U recovers A.

Step 2: Forward substitution

$$\mathbf{Ly} = \mathbf{b}$$

$$y_1 = 1, \qquad 2y_1 + y_2 = 3$$

Step 3: Back substitution

$$\mathbf{Ux} = \mathbf{y}$$

$$x_2 = 1, \qquad 2x_1 + x_2 = 1$$

$$x_1 = 0, \qquad x_2 = 1$$

QR Example: Constructing Orthonormal Columns

Step 1: Normalize the first column

$$\mathbf{A} = \begin{bmatrix} 1 & 0 \\ 1 & 2 \end{bmatrix}, \mathbf{a}_1 = \begin{bmatrix} 1 \\ 1 \end{bmatrix}$$

$$\mathbf{q}_1 = \left(\frac{1}{\sqrt{2}}\right)\begin{bmatrix} 1 \\ 1 \end{bmatrix}$$

$$r_{11} = \|\mathbf{a}_1\| = \sqrt{2}$$

Step 2: Remove the parallel component

$$r_{12} = \mathbf{q}_1^{\mathrm{T}}\mathbf{a}_2 = \sqrt{2}$$

$$\mathbf{u}_2 = \mathbf{a}_2 - r_{12}\mathbf{q}_1 = \begin{bmatrix} -1 \\ 1 \end{bmatrix}$$

$$\mathbf{q}_2 = \left(\frac{1}{\sqrt{2}}\right)\begin{bmatrix} -1 \\ 1 \end{bmatrix}$$

The two q vectors have length 1 and dot product 0. The remaining norm gives $r_{22} = \sqrt{2}$.

QR Example: Solve Using the Triangular Factor

Step 3: Assemble and check the factors

$$\mathbf{Q} = \left(\frac{1}{\sqrt{2}}\right)\begin{bmatrix} 1 & -1 \\ 1 & 1 \end{bmatrix}$$

$$\mathbf{R} = \begin{bmatrix} \sqrt{2} & \sqrt{2} \\ 0 & \sqrt{2} \end{bmatrix}$$

$$\mathbf{Q}^{\mathrm{T}}\mathbf{Q} = \mathbf{I}, \qquad \mathbf{QR} = \mathbf{A}$$

Step 4: Solve for b = (1, 3)

$$\mathbf{Q}^{\mathrm{T}}\mathbf{b} = \begin{bmatrix} 2\sqrt{2} \\ \sqrt{2} \end{bmatrix}$$

$$\sqrt{2}x_2 = \sqrt{2}$$

$$\sqrt{2}x_1 + \sqrt{2}x_2 = 2\sqrt{2}$$

$$x_1 = 1, \qquad x_2 = 1$$

Check in the original equations: 1 = 1 and 1 + 2 = 3.

Choosing a Matrix Decomposition



Match the method to the problem

Cholesky: symmetric positive definite systems. About $n^3/3$ floating-point operations.

QR: full-column-rank least squares. Householder QR costs about $2mn^2 - 2n^3/3$ for $m \geq n$.

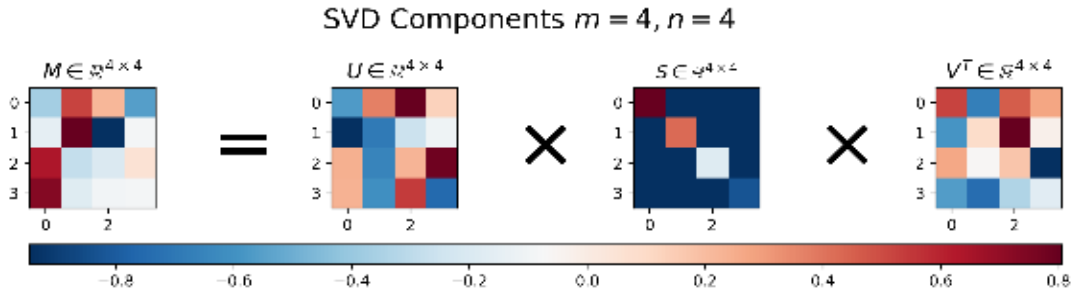
SVD: rank, null spaces and ill-conditioned least squares. Cost depends on the algorithm and whether singular vectors are required.

Pivoted LU: general nonsingular square systems. Reuse the factors when solving for several right-hand sides.

“flops” counts floating-point operations. FLOP/s measures speed. Compare equal output requirements and the same matrix data.

SVD: Matrix Entries and Factor Structure

The original colour illustration shows how the factors represent different parts of a matrix.



Colours encode entries, not matrix dimensions. The diagonal factor contains nonnegative singular values. U and V contain orthonormal directions.

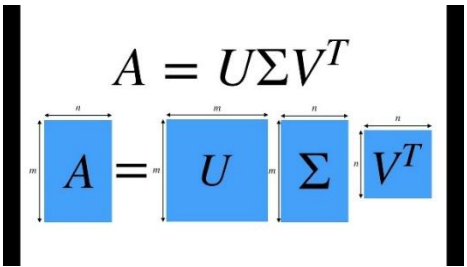
Singular Value Decomposition - I: The Theorem

The SVD Theorem

- Any matrix $\mathbf{A} \in \mathbb{R}^{m \times n}$ can be factored as:

$$\mathbf{A} = \mathbf{U}\mathbf{\Sigma}\mathbf{V}^{\mathrm{T}}$$

- Where:
  - $\mathbf{U} \in \mathbb{R}^{m \times m}$ is orthogonal $\mathbf{U}^{\mathrm{T}}\mathbf{U} = \mathbf{I}$
  - $\mathbf{\Sigma} \in \mathbb{R}^{m \times n}$ is diagonal with non-negative entries
  - $\mathbf{V} \in \mathbb{R}^{n \times n}$ is orthogonal $\mathbf{V}^{\mathrm{T}}\mathbf{V} = \mathbf{I}$



- Diagonal entries σi of $\mathbf{\Sigma}$ are the singular values of $\mathbf{A}$
- Singular values are arranged in descending order: $\sigma_1 \geq \sigma_2 \geq \cdots \geq \sigma_r > 0$
- Number of non-zero singular values equals the rank of $\mathbf{A}$
- The columns of $\mathbf{U}$ are called left singular vectors
- The columns of $\mathbf{V}$ are called right singular vectors

Full SVD: Dimensions and Rank-One Terms

$$\mathbf{A} = \mathbf{U}\boldsymbol{\Sigma}\mathbf{V}^{\mathrm{T}}$$

Full factor dimensions

$\mathbf{A} \in \mathbb{R}^{m \times n}$

$\mathbf{U} \in \mathbb{R}^{m \times m}$

$\boldsymbol{\Sigma} \in \mathbb{R}^{m \times n}$

$\mathbf{V} \in \mathbb{R}^{n \times n}$

One input direction at a time

$$\mathbf{A}\mathbf{v}_i = \sigma_i \mathbf{u}_i$$

$$\mathbf{A} = \sum_{i=1}^{r} \sigma_i \mathbf{u}_i \mathbf{v}_i^{\mathrm{T}}$$

Here r is the rank. Each outer product contributes one independent input-to-output direction, scaled by its singular value.

SVD Example Calculation - Part 1

Example: Find the SVD of matrix A

$$\mathbf{A} = \begin{bmatrix} 2 & 2 \\ 1 & 3 \\ 3 & 1 \end{bmatrix}$$

$$\textbf{The rank of } \mathbf{A} \text{ is } 2$$

Step 1: Calculate $\mathbf{A}^{\mathrm{T}}\mathbf{A}$ and $\mathbf{A}\mathbf{A}^{\mathrm{T}}$

$$\mathbf{A}^{\mathrm{T}}\mathbf{A} = \begin{bmatrix} 2 & 1 & 3 \\ 2 & 3 & 1 \end{bmatrix} \begin{bmatrix} 2 & 2 \\ 1 & 3 \\ 3 & 1 \end{bmatrix}$$

$$= \begin{bmatrix} 4+1+9 & 4+3+3 \\ 4+3+3 & 4+9+1 \end{bmatrix}$$

$$= \begin{bmatrix} 14 & 10 \\ 10 & 14 \end{bmatrix}$$

Similarly:

$$\mathbf{A}\mathbf{A}^{\mathrm{T}} = \begin{bmatrix} 8 & 8 & 8 \\ 8 & 10 & 6 \\ 8 & 6 & 10 \end{bmatrix}$$

Step 2: Find eigenvalues of $\mathbf{A}^{T}\mathbf{A}$

$$\det\left(\mathbf{A}^{\mathrm{T}}\mathbf{A} - \lambda\mathbf{I}\right) = 0$$

$$\det\begin{bmatrix} 14-\lambda & 10 \\ 10 & 14-\lambda \end{bmatrix} = 0$$

$$(14-\lambda)^2 - 100 = 0$$

$$\lambda^2 - 28\lambda + 96 = 0$$

$$(\lambda - 24)(\lambda - 4) = 0$$

So $\lambda_1 = 24$ and $\lambda_2 = 4$

Step 3: Calculate singular values

$$\sigma_i = \sqrt{\lambda_i}$$

Therefore:

$$\sigma_1 = \sqrt{24} = 2\sqrt{6}$$

$$\sigma_2 = \sqrt{4} = 2$$

Why the First Example Has Rank Two

$$\mathbf{A} = \begin{bmatrix} 2 & 2 \\ 1 & 3 \\ 3 & 1 \end{bmatrix}$$

There are two columns, so the rank cannot exceed two.

The first two rows give a nonsingular 2 × 2 minor:

$$2 \times 3 - 2 \times 1 = 4 \neq 0$$

The two input directions are independent

Neither column is a multiple of the other. Thus both input components produce distinguishable measurement changes.

$$\mathbf{A} \to \begin{bmatrix} 1 & 1 \\ 0 & 2 \\ 0 & -2 \end{bmatrix} \to \begin{bmatrix} 1 & 0 \\ 0 & 1 \\ 0 & 0 \end{bmatrix}$$

Two pivots confirm rank two. Both singular values are positive.

SVD Example 1: Singular Vectors

Step 4: Right singular vectors

$$(\mathbf{A}^{\mathrm{T}}\mathbf{A} - 24\mathbf{I}) = \begin{bmatrix} -10 & 10 \\ 10 & -10 \end{bmatrix}$$

The two components are equal. Normalize to unit length:

$$\mathbf{v}_1 = \left(\frac{1}{\sqrt{2}}\right)\begin{bmatrix} 1 \\ 1 \end{bmatrix}$$

$$(\mathbf{A}^{\mathrm{T}}\mathbf{A} - 4\mathbf{I}) = \begin{bmatrix} 10 & 10 \\ 10 & 10 \end{bmatrix}$$

$$\mathbf{v}_2 = \left(\frac{1}{\sqrt{2}}\right)\begin{bmatrix} 1 \\ -1 \end{bmatrix}$$

Step 5: Left singular vectors

$$\mathbf{u}_i = \mathbf{A}\frac{\mathbf{v}_i}{\sigma_i}$$

$$\mathbf{u}_1 = \left(\frac{1}{\sqrt{3}}\right)\begin{bmatrix} 1 \\ 1 \\ 1 \end{bmatrix}$$

$$\mathbf{u}_2 = \left(\frac{1}{\sqrt{2}}\right)\begin{bmatrix} 0 \\ -1 \\ 1 \end{bmatrix}$$

A^TA is 2 × 2 and AA^T is 3 × 3. They share nonzero eigenvalues, but their eigenvectors have different dimensions.

SVD Example 1: Complete and Verify

Complete the orthonormal output basis

$\mathbf{u}_3^{\mathrm{T}}\mathbf{u}_1 = 0, \qquad \mathbf{u}_3^{\mathrm{T}}\mathbf{u}_2 = 0$

$u_{31} + u_{32} + u_{33} = 0$

$-u_{32} + u_{33} = 0$

$u_{31}^2 + u_{32}^2 + u_{33}^2 = 1$

$$\mathbf{u}_3 = \left(\frac{1}{\sqrt{6}}\right)\begin{bmatrix}-2\\1\\1\end{bmatrix}$$

Either sign of this last column is valid.

Assemble the full factors

$\mathbf{U} = [\mathbf{u}_1, \mathbf{u}_2, \mathbf{u}_3]$

$$\mathbf{\Sigma} = \begin{bmatrix}2\sqrt{6} & 0\\0 & 2\\0 & 0\end{bmatrix}$$

$$\mathbf{V}^{\mathrm{T}} = \left(\frac{1}{\sqrt{2}}\right)\begin{bmatrix}1 & 1\\1 & -1\end{bmatrix}$$

$\mathbf{A} = 2\sqrt{6}\mathbf{u}_1\mathbf{v}_1^{\mathrm{T}} + 2\mathbf{u}_2\mathbf{v}_2^{\mathrm{T}}$

The zero third row of Σ makes the choice of sign for the last column of U irrelevant to reconstruction.

Class Exercise: SVD of a Measurement Matrix

A simplified 2D measurement model

$$\mathbf{A} = \begin{bmatrix} 2 & 2 \\ 1 & 3 \\ 3 & 1 \end{bmatrix}$$

Each row represents a measurement. The columns correspond to East and North position components.

1. Compute A^TA and AA^T.
2. Find eigenvalues of A^TA and the singular values.
3. Normalize the right singular vectors.
4. Calculate matching left singular vectors.
5. Construct the full and compact decompositions.
6. Verify reconstruction, orthogonality and rank.

Use the previous example to check your work. Changing the sign of one singular vector requires changing its paired vector too.

SVD Example Calculation - Part 2

Example: Find the SVD of matrix A

$$\mathbf{A} = \begin{bmatrix} 1 & 2 \\ 2 & 4 \\ 3 & 6 \end{bmatrix}$$

The rank of **A** is 1

Step 1: Calculate $\mathbf{A}^{\mathrm{T}}\mathbf{A}$ and $\mathbf{A}\mathbf{A}^{\mathrm{T}}$

$$\mathbf{A}^{\mathrm{T}}\mathbf{A} = \begin{bmatrix} 1 & 2 & 3 \\ 2 & 4 & 6 \end{bmatrix}\begin{bmatrix} 1 & 2 \\ 2 & 4 \\ 3 & 6 \end{bmatrix}$$

$$= \begin{bmatrix} 1+4+9 & 2+8+18 \\ 2+8+18 & 4+16+36 \end{bmatrix}$$

$$= \begin{bmatrix} 14 & 28 \\ 28 & 56 \end{bmatrix}$$

Similarly:

$$\mathbf{A}\mathbf{A}^{\mathrm{T}} = \begin{bmatrix} 5 & 10 & 15 \\ 10 & 20 & 30 \\ 15 & 30 & 45 \end{bmatrix}$$

Step 2: Find eigenvalues of $\mathbf{A}^{T}\mathbf{A}$

$$\det\left(\mathbf{A}^{\mathrm{T}}\mathbf{A} - \lambda\mathbf{I}\right) = 0$$

$$\det\begin{bmatrix} 14-\lambda & 28 \\ 28 & 56-\lambda \end{bmatrix} = 0$$

$$(14-\lambda)(56-\lambda) - 784 = 0$$

$$\lambda^2 - 70\lambda = 0$$

$$(\lambda - 70)\lambda = 0$$

So $\lambda_1 = 70$ and $\lambda_2 = 0$

Step 3: Calculate singular values

$$\sigma_i = \sqrt{\lambda_i}$$

Therefore:

$$\sigma_1 = \sqrt{70}$$

$$\sigma_2 = 0$$

Why the Second Example Has Rank One

$$\mathbf{A} = \begin{bmatrix} 1 & 2 \\ 2 & 4 \\ 3 & 6 \end{bmatrix}$$

Column 2 is twice column 1. All rows are multiples of the first row.

$$\mathbf{A} \rightarrow \begin{bmatrix} 1 & 2 \\ 0 & 0 \\ 0 & 0 \end{bmatrix}$$

Only one pivot remains.

Only one input combination is observed

$$\mathbf{A}\begin{bmatrix} x_1 \\ x_2 \end{bmatrix} = \begin{bmatrix} 1 \\ 2 \\ 3 \end{bmatrix}(x_1 + 2x_2)$$

Different inputs with the same combination give identical outputs.

$$\mathbf{A}\begin{bmatrix} 2 \\ -1 \end{bmatrix} = \begin{bmatrix} 0 \\ 0 \\ 0 \end{bmatrix}$$

This null direction has singular value zero.

SVD Example 2: Rank-Deficient Directions

Step 4: Input directions

$$\mathbf{A}^{\mathrm{T}}\mathbf{A} = \begin{bmatrix} 14 & 28 \\ 28 & 56 \end{bmatrix}$$

$$\lambda_1 = 70, \qquad \lambda_2 = 0$$

$$\mathbf{v}_1 = \left(\frac{1}{\sqrt{5}}\right)\begin{bmatrix} 1 \\ 2 \end{bmatrix}$$

$$\mathbf{v}_2 = \left(\frac{1}{\sqrt{5}}\right)\begin{bmatrix} 2 \\ -1 \end{bmatrix}$$

The second input direction lies in the null space: its image is zero.

Step 5: Output directions

$$\mathbf{u}_1 = \mathbf{A}\frac{\mathbf{v}_1}{\sqrt{70}} = \left(\frac{1}{\sqrt{14}}\right)\begin{bmatrix} 1 \\ 2 \\ 3 \end{bmatrix}$$

The zero singular value cannot be used as a divisor. Complete U with orthonormal columns:

$$\mathbf{u}_2 = \left(\frac{1}{\sqrt{5}}\right)\begin{bmatrix} 2 \\ -1 \\ 0 \end{bmatrix}$$

$$\mathbf{u}_3 = \left(\frac{1}{\sqrt{70}}\right)\begin{bmatrix} 3 \\ 6 \\ -5 \end{bmatrix}$$

SVD Example 2: Full and Compact Factors

Full factorization

$$\mathbf{U} = [\mathbf{u}_1, \mathbf{u}_2, \mathbf{u}_3]$$

$$\mathbf{\Sigma} = \begin{bmatrix} \sqrt{70} & 0 \\ 0 & 0 \\ 0 & 0 \end{bmatrix}$$

$$\mathbf{V}^{\mathrm{T}} = \left(\frac{1}{\sqrt{5}}\right)\begin{bmatrix} 1 & 2 \\ 2 & -1 \end{bmatrix}$$

$$\mathbf{U}^{\mathrm{T}}\mathbf{U} = \mathbf{I}, \qquad \mathbf{V}^{\mathrm{T}}\mathbf{V} = \mathbf{I}$$

Only one rank-one term survives

$$\mathbf{A} = \sqrt{70}\,\mathbf{u}_1\mathbf{v}_1^{\mathrm{T}}$$

$$\mathbf{A} = \sqrt{70}\left(\frac{1}{\sqrt{14}}\right)\begin{bmatrix} 1 \\ 2 \\ 3 \end{bmatrix}\left(\frac{1}{\sqrt{5}}\right)[1 \quad 2]$$

$$\mathbf{A} = \begin{bmatrix} 1 & 2 \\ 2 & 4 \\ 3 & 6 \end{bmatrix}$$

The extra columns of U complete the basis. Only the first singular pair contributes to A.

Singular Value Decomposition - III: Interpretation

Geometric interpretation
V^T: orthogonal input transformation
Σ: scaling along principal axes
U: orthogonal output transformation
Orthogonal factors may include reflections.

Transformation Sequence

First apply V^T to the input coordinates.
Then scale by the singular values.
Finally apply U in the output space.

Key Properties

- SVD reveals principal directions and stretching factors of the transformation
- SVD always exists for any matrix (unlike eigendecomposition)
- Useful for low-rank approximations:

$$\mathbf{A}_k = \sum_{i=1}^{k} \sigma_i \mathbf{u}_i \mathbf{v}_i^{\mathrm{T}}$$

- Basis for Principal Component Analysis
- Essential for pseudo-inverse calculation:

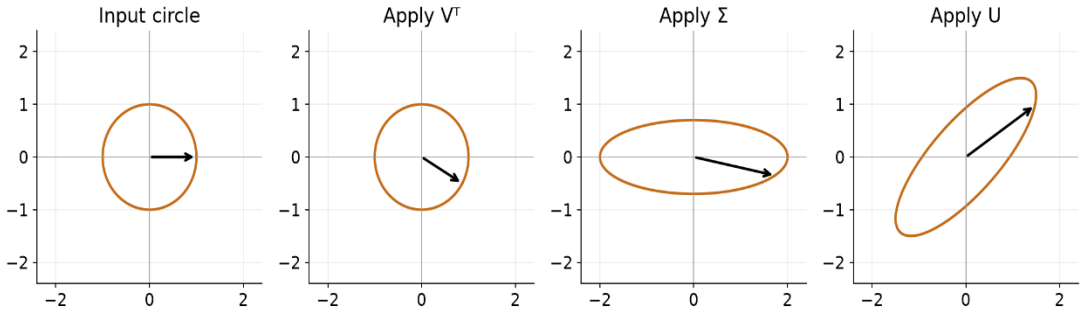
$$\mathbf{A}^{+} = \mathbf{V}\boldsymbol{\Sigma}^{+}\mathbf{U}^{\mathrm{T}}$$

- Helps solve least-squares problems
- Powerful tool for noise reduction in data

The singular values σ_i indicate the importance of each principal direction in the transformation. The largest singular values correspond to the most significant components.

SVD Geometry: Rotation, Scaling and Rotation

An orthogonal factor can rotate or reflect. The middle factor stretches the principal directions.



Read right to left in the matrix product. The unit circle becomes an ellipse whose semi-axis lengths are the singular values.

Compact SVD and Low-Rank Approximation



Keep the r positive singular values

$$\mathbf{A} = \mathbf{U}_r \mathbf{\Sigma}_r \mathbf{V}_r^{\mathrm{T}}$$

$$\mathbf{U}_r \in \mathbb{R}^{m \times r}, \qquad \mathbf{\Sigma}_r \in \mathbb{R}^{r \times r}, \qquad \mathbf{V}_r \in \mathbb{R}^{n \times r}$$

The columns of the two basis matrices are orthonormal. The square diagonal factor contains only positive singular values.

$$\mathbf{A}_k = \sum_{i=1}^{k} \sigma_i \mathbf{u}_i \mathbf{v}_i^{\mathrm{T}}$$

For k < r, this is a best rank-k approximation in the Frobenius and spectral norms. Truncation saves storage and discards weak directions.

SVD Properties and Matrix Norms

Connection to eigenvalues

$$\mathbf{A}^{\mathrm{T}}\mathbf{A}\mathbf{v}_i = \sigma_i^2\mathbf{v}_i$$

$$\mathbf{A}\mathbf{A}^{\mathrm{T}}\mathbf{u}_i = \sigma_i^2\mathbf{u}_i$$

The two Gram matrices share nonzero eigenvalues. Their dimensions and zero-eigenvalue multiplicities may differ.

For a real symmetric matrix, singular values equal the absolute eigenvalues, arranged in descending order.

Frobenius norm

$$\|\mathbf{A}\|_F = \sqrt{\sum_{i=1}^{r}\sigma_i^2}$$

Spectral norm

$$\|\mathbf{A}\|_2 = \sigma_1$$

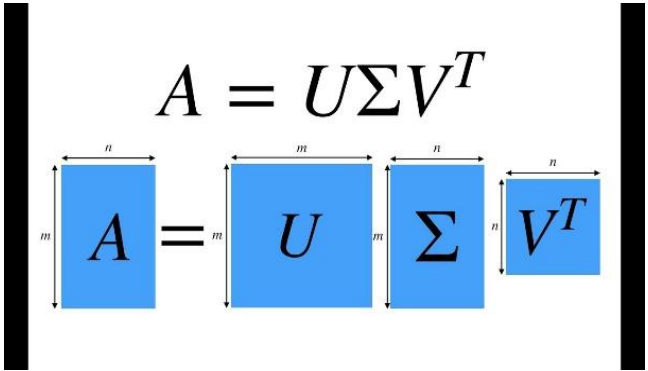
Condition number: square nonsingular A

$$\kappa_2(\mathbf{A}) = \frac{\sigma_1}{\sigma_n}$$

Nuclear norm

$$\|\mathbf{A}\|_* = \sum_{i=1}^{r}\sigma_i$$

SVD Dimensions: Understanding the Original Diagram



This diagram shows the full SVD. Check each shared dimension in the product.

A: m × n
U: m × m
Σ: m × n
V^T: n × n

Both U and V are square orthogonal matrices. The diagonal factor can be rectangular.

For m ≥ n, thin SVD uses U of size m × n and Σ of size n × n. Compact SVD retains only the r positive singular values.

SVD Example Calculation - Part 3

Example: Find the SVD of matrix A

$$\mathbf{A} = \begin{bmatrix} 3 & 2 \\ 2 & 3 \end{bmatrix}$$

Step 1: Calculate $\mathbf{A}^{\mathrm{T}}\mathbf{A}$ and $\mathbf{A}\mathbf{A}^{\mathrm{T}}$

$$\mathbf{A}^{\mathrm{T}}\mathbf{A} = \begin{bmatrix} 3 & 2 \\ 2 & 3 \end{bmatrix}\begin{bmatrix} 3 & 2 \\ 2 & 3 \end{bmatrix}$$
$$= \begin{bmatrix} 9+4 & 6+6 \\ 6+6 & 4+9 \end{bmatrix}$$
$$= \begin{bmatrix} 13 & 12 \\ 12 & 13 \end{bmatrix}$$

Similarly:

$$\mathbf{A}\mathbf{A}^{\mathrm{T}} = \begin{bmatrix} 13 & 12 \\ 12 & 13 \end{bmatrix}$$

Step 2: Find eigenvalues of $\mathbf{A}^{T}\mathbf{A}$

$$\det\left(\mathbf{A}^{\mathrm{T}}\mathbf{A} - \lambda\mathbf{I}\right) = 0$$
$$\det\begin{bmatrix} 13-\lambda & 12 \\ 12 & 13-\lambda \end{bmatrix} = 0$$
$$(13-\lambda)^2 - 12^2 = 0$$
$$(13-\lambda)^2 = 144$$
$$13-\lambda = \pm 12$$

So $\lambda_1 = 25$ and $\lambda_2 = 1$

Step 3: Calculate singular values

$$\sigma_i = \sqrt{\lambda_i}$$

Therefore:

$$\sigma_1 = \sqrt{25} = 5$$
$$\sigma_2 = \sqrt{1} = 1$$

SVD Example Calculation - Part 3

Step 4: Find right singular vectors (eigenvectors of $\mathbf{A}^{\mathrm{T}}\mathbf{A}$)

For $\lambda_1 = 25$:

$$(\mathbf{A}^{\mathrm{T}}\mathbf{A} - 25\mathbf{I})\mathbf{v}_1 = 0$$

$$\begin{bmatrix} -12 & 12 \\ 12 & -12 \end{bmatrix}\mathbf{v}_1 = 0$$

This gives us: $-12v_{11} + 12v_{12} = 0$,so $v_{11} = v_{12}$

Normalizing: $\mathbf{v}_1 = \frac{1}{\sqrt{2}}\begin{bmatrix} 1 \\ 1 \end{bmatrix}$

For $\lambda_2 = 1$:

$$(\mathbf{A}^{\mathrm{T}}\mathbf{A} - 1\mathbf{I})\mathbf{v}_2 = 0$$

$$\begin{bmatrix} 12 & 12 \\ 12 & 12 \end{bmatrix}\mathbf{v}_2 = 0$$

This gives us: $v_{21} = -v_{22}$

Normalizing: $\mathbf{v}_2 = \frac{1}{\sqrt{2}}\begin{bmatrix} 1 \\ -1 \end{bmatrix}$

Step 5: Calculate left singular vectors Method 1: Find eigenvectors of $\mathbf{A}\mathbf{A}^{\mathrm{T}}$)same as $\mathbf{A}^{\mathrm{T}}\mathbf{A}$ in this case)

$$\mathbf{u}_1 = \frac{1}{\sqrt{2}}\begin{bmatrix} 1 \\ 1 \end{bmatrix} and\ \mathbf{u}_2 = \frac{1}{\sqrt{2}}\begin{bmatrix} 1 \\ -1 \end{bmatrix}$$

Method 2: Using the formula $\mathbf{u}_i = \frac{1}{\sigma_i}\mathbf{A}\mathbf{v}_i$

$$\begin{aligned} \mathbf{u}_1 &= \frac{1}{5}\mathbf{A}\mathbf{v}_1 \\ &= \frac{1}{5}\begin{bmatrix} 3 & 2 \\ 2 & 3 \end{bmatrix}\frac{1}{\sqrt{2}}\begin{bmatrix} 1 \\ 1 \end{bmatrix} \\ &= \frac{1}{5\sqrt{2}}\begin{bmatrix} 5 \\ 5 \end{bmatrix} \\ &= \frac{1}{\sqrt{2}}\begin{bmatrix} 1 \\ 1 \end{bmatrix} \end{aligned}$$

Similar calculation confirms $\mathbf{u}_2 = \frac{1}{\sqrt{2}}\begin{bmatrix} 1 \\ -1 \end{bmatrix}$

SVD Approximation: Keeping One Singular Mode

Given the symmetric example

$$\mathbf{A} = \begin{bmatrix} 3 & 2 \\ 2 & 3 \end{bmatrix}$$

$$\sigma_1 = 5, \qquad \sigma_2 = 1$$

$$\mathbf{u}_1 = \mathbf{v}_1 = \left(\frac{1}{\sqrt{2}}\right)\begin{bmatrix} 1 \\ 1 \end{bmatrix}$$

Step 1: Keep the larger singular value

$$\mathbf{A}_1 = 5\mathbf{u}_1\mathbf{v}_1^{\mathrm{T}}$$

$$\mathbf{A}_1 = \left(\frac{5}{2}\right)\begin{bmatrix} 1 & 1 \\ 1 & 1 \end{bmatrix}$$

$$\mathbf{A}_1 = \begin{bmatrix} 2.5 & 2.5 \\ 2.5 & 2.5 \end{bmatrix}$$

The rank-one approximation keeps the shared component of the two columns.

SVD Approximation: Quantifying What Was Lost

Step 2: Compute the residual matrix

$$\mathbf{E} = \mathbf{A} - \mathbf{A}_1 = \begin{bmatrix} 0.5 & -0.5 \\ -0.5 & 0.5 \end{bmatrix}$$

$$\|\mathbf{E}\|_F = \sqrt{4 \times 0.5^2} = 1$$

$$\|\mathbf{A}\|_F = \sqrt{26} \approx 5.10$$

Step 3: Relative reconstruction error

$$\frac{\|\mathbf{E}\|_F}{\|\mathbf{A}\|_F} = \frac{1}{\sqrt{26}} \approx 19.61\%$$

The error equals the discarded singular value here: only one mode was dropped.

Try retaining both modes. The reconstructed matrix then matches A, up to numerical rounding.

SVD in Navigation and Coordinate Transformations - I

Coordinate Transformation Context
When $\mathbf{A}$ represents a transformation matrix:

- $\mathbf{V}$ identifies principal input directions
- $\mathbf{\Sigma}$ quantifies distortion along each direction
- $\mathbf{U}$ shows where those directions map to

Comparison with Eigendecomposition

Rigid transformations
A rotation has every singular value equal to one.
This checks orthogonality but does not rule out reflection.
A proper rotation also has determinant +1.
The nearest-rotation construction needs a determinant correction.

Applications
Magnetometer soft-iron calibration
IMU scale and cross-axis calibration
Camera geometry

Eigendecomposition	SVD
$\mathbf{A} = \mathbf{Q}\mathbf{\Lambda}\mathbf{Q}^{-1}$	$\mathbf{A} = \mathbf{U}\mathbf{\Sigma}\mathbf{V}^{\mathrm{T}}$
Requires square matrix	Works for any matrix
May not exist	Always exists
Same input/output space	Can connect different spaces

SVD in Navigation and Coordinate Transformations - II

GNSS position estimation
Small singular values of the weighted geometry matrix indicate weakly observed directions.
SVD supports least-squares solutions and regularization.
For isotropic measurement noise, position standard deviations vary inversely with geometry singular values.
Right singular vectors give the corresponding state directions.

2. **Map/Terrain Matching**
   - Finding optimal alignment between maps
   - Extracting principal terrain features
   - Reducing dimensionality of terrain data

3. **INS Mechanization**
   - Stable integration of rotation matrices
   - Detecting and correcting numerical errors
   - Analysis of error propagation in inertial solutions
   - Correcting accumulated drift in attitude matrices

Key Course Connection

- SVD provides mathematical tools to understand linear transformations in navigation
- Essential for robust implementation of navigation algorithms
- Bridges mathematical theory with practical sensor processing

Least Squares: A Complete Numerical Example

Data: fit one value to two measurements

$$\mathbf{A} = \begin{bmatrix} 1 \\ 1 \end{bmatrix}, \mathbf{b} = \begin{bmatrix} 1 \\ 3 \end{bmatrix}$$

The equations x = 1 and x = 3 cannot both hold.
Minimize the sum of squared residuals.

$$g(x) = (x-1)^2 + (x-3)^2$$

Calculation and check

$$g'(x) = 2(x-1) + 2(x-3) = 0$$

$$4x - 8 = 0, \qquad x = 2$$

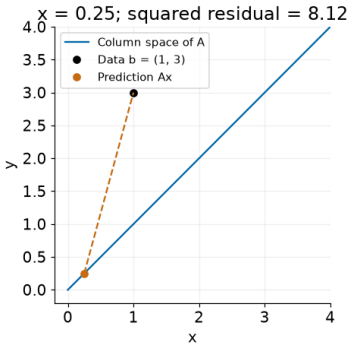
$$\mathbf{r} = \mathbf{b} - \mathbf{A}x = \begin{bmatrix} -1 \\ 1 \end{bmatrix}$$

$$\mathbf{A}^{\mathrm{T}}\mathbf{r} = 0, \qquad \|\mathbf{r}\|^2 = 2$$

The residual is perpendicular to the column space. The pseudoinverse returns the same least-squares solution.

Least Squares Animation: The Closest Prediction

The two predicted measurements are both x. Therefore every possible prediction lies on the line y = x.



The measured data are (1, 3), which are off the prediction line.

As x approaches 2, the squared residual decreases to its minimum of 2.

At the closest point, the residual is perpendicular to the line.

$$\mathbf{A} = \begin{bmatrix} 1 \\ 1 \end{bmatrix}, \mathbf{b} = \begin{bmatrix} 1 \\ 3 \end{bmatrix}, \mathbf{A}x = \begin{bmatrix} x \\ x \end{bmatrix}$$

Weighted Least Squares: Unequal Measurement Quality



Data: the second sensor is less precise

Measurements: 1 m and 3 m.
Standard deviations: 1 m and 3 m.
Assume independent, unbiased errors.

$$g(x) = (x-1)^2 + \frac{(x-3)^2}{9}$$

Weight each squared residual by the inverse measurement variance.

Calculation

$$2(x-1) + \left(\frac{2}{9}\right)(x-3) = 0$$

$$9x - 9 + x - 3 = 0$$

$$10x = 12, \qquad x = 1.20$$

The answer lies closer to the more precise sensor. Equal weights would give 2.00 m.

SVD for Linear Systems - II

Handling Ill-Conditioned Systems

- For small singular values, reciprocals become very large
- Can lead to numerical instability when $\sigma_i \approx 0$
- Solutions:
  - — **Truncated SVD:** Set all $\sigma_i <$ threshold to zero
  - — **Tikhonov regularization:** Replace σ_i^{-1} with $\frac{\sigma_i}{\sigma_i^2+\alpha^2}$
- Regularized solution:

$$x_\alpha = \sum_{i=1}^{r} \frac{\sigma_i}{\sigma_i^2 + \alpha^2} \left(\mathbf{u}_i^{\mathrm{T}} \mathbf{b}\right) \mathbf{v}_i$$

where α is the regularization parameter

Navigation applications
Poor GNSS geometry creates weakly observed state directions.
Truncation suppresses small singular modes, not individual satellites.
Regularization trades sensitivity against bias.

Sensor calibration
Diagnose insufficient excitation.
Identify scale or bias combinations that the measurements cannot separate.

SVD Example 4: A Nonsymmetric Matrix

Step 1: Form the Gram matrix

$$\mathbf{A} = \begin{bmatrix} 4 & 0 \\ 3 & -5 \end{bmatrix}$$

$$\mathbf{A}^{\mathrm{T}}\mathbf{A} = \begin{bmatrix} 25 & -15 \\ -15 & 25 \end{bmatrix}$$

Step 2: Eigenvalues and singular values

$$\lambda_1 = 40, \qquad \lambda_2 = 10$$

$$\sigma_1 = \sqrt{40} \approx 6.32$$

$$\sigma_2 = \sqrt{10} \approx 3.16$$

Step 3: Normalize the input directions

$$\mathbf{V} = \left(\frac{1}{\sqrt{2}}\right)\begin{bmatrix} 1 & 1 \\ -1 & 1 \end{bmatrix}$$

Step 4: Match each output direction

$$\mathbf{u}_i = \mathbf{A}\frac{\mathbf{v}_i}{\sigma_i}$$

$$\mathbf{U} = \left(\frac{1}{\sqrt{5}}\right)\begin{bmatrix} 1 & 2 \\ 2 & -1 \end{bmatrix}$$

$$\mathbf{A} = \mathbf{U}\mathrm{diag}\left(\sqrt{40}, \sqrt{10}\right)\mathbf{V}^{\mathrm{T}}$$

Quick Quiz: SVD Properties

1. Which statement is always true about singular values?
   A) They are nonnegative.
   B) They are square roots of the eigenvalues of A.
   C) They are all nonzero for any rectangular matrix.
   D) They equal the eigenvalues of A.

2. What does a condition number describe?
   A) The exact numerical error.
   B) Sensitivity to perturbations in the data.
   C) Whether a matrix is symmetric.
   D) The execution time of the solver.

3. Why is SVD useful in navigation?
   A) Every system has a unique exact solution.
   B) It handles rectangular matrices and reveals rank deficiency.
   C) It is always the fastest method.
   D) It always uses the least memory.

Quick Quiz: SVD Properties - Solutions



Question 1: Answer A
Singular values are nonnegative.
They are square roots of the eigenvalues of A^TA.
There are min(m, n) singular values, including zeros.
The number of nonzero values equals the rank.

Question 2: Answer B
The condition number describes sensitivity to data perturbations.
It bounds possible error amplification under the stated assumptions.
A sensitive problem can still be solved with a numerically stable algorithm.

Quick Quiz: SVD Properties - Solutions (cont.)

3. What makes SVD particularly valuable for navigation problems?
   Answer: B) It works for any matrix shape and reveals rank deficiencies
   - SVD applies to any rectangular matrix, unlike eigendecomposition
   - It immediately reveals rank deficiency through zero singular values
   - In GNSS positioning, rank deficiency indicates insufficient satellite geometry
   - It provides a solution that minimizes error even in underdetermined systems
   - SVD handles both overdetermined systems (more measurements than unknowns) and underdetermined systems (fewer measurements than unknowns)

SVD reveals poorly observed directions. A tolerance or regularization rule is still needed when singular values are very small.

Differentiation: Rules and Domains

Power and chain rules

$$\frac{d}{dx}(x^n) = nx^{n-1}$$

$$\frac{d}{dx}f\big(g(x)\big) = f'\big(g(x)\big)g'(x)$$

Product and quotient rules

$$(uv)' = u'v + uv'$$

$$\left(\frac{u}{v}\right)' = \frac{u'v - uv'}{v^2}$$

The quotient requires a nonzero denominator.

Common derivatives

$$\frac{d}{dx}\sin x = \cos x$$

$$\frac{d}{dx}\cos x = -\sin x$$

$$\frac{d}{dx}e^{\mathrm{x}} = e^{\mathrm{x}}$$

$$\frac{d}{dx}\ln x = \frac{1}{x}$$

$$\frac{d}{dx}\sqrt{x} = \frac{1}{2\sqrt{x}}$$

Radians. Log and square root: x > 0.

Differentiation Review: Navigation Applications

Example 1: Range Function

Distance to a fixed point:

$$r = \sqrt{(x - x_0)^2 + (y - y_0)^2}$$

Time derivative (using chain rule):

$$\frac{dr}{dt} = \frac{(x - x_0)\dot{x} + (y - y_0)\dot{y}}{r}$$

Physical meaning:

- Radial velocity component
- Positive: moving away
- Negative: approaching

Example 2: Position to Velocity

Given position:

$$x(t) = v_0 t + \frac{1}{2} a t^2$$

Velocity:

$$v(t) = \frac{dx}{dt} = v_0 + at$$

Example 3: 3D Satellite Range

Distance to satellite:

$$\rho = \sqrt{(x - x_s)^2 + (y - y_s)^2 + (z - z_s)^2}$$

Partial derivatives:

$$\frac{\partial \rho}{\partial x} = \frac{x - x_s}{\rho}$$

$$\frac{\partial \rho}{\partial y} = \frac{y - y_s}{\rho}$$

$$\frac{\partial \rho}{\partial z} = \frac{z - z_s}{\rho}$$

Gradient vector:

$$\nabla \rho = \frac{1}{\rho} \begin{bmatrix} x - x_s \\ y - y_s \\ z - z_s \end{bmatrix}$$

Practice: Range, Bearing and Doppler

1. Satellite range

Satellite coordinates are (20000, 15000, 25000) km. Express receiver coordinates in km too, and differentiate the range with respect to each coordinate.

$$d = \sqrt{(x - 20000)^2 + (y - 15000)^2 + (z - 25000)^2}$$

2. Bearing clockwise from North

Let x point East and y point North. Find the derivatives with respect to the starting position.

$$\beta = \operatorname{atan2}(x_2 - x_1, y_2 - y_1)$$

3. Doppler shift: stationary satellite, moving receiver

$$f_{\mathrm{D}} = f_{\mathrm{received}} - f_0 = -\frac{f_0}{c}\frac{\mathrm{d}d}{\mathrm{d}t}$$

Practice Solutions: Signs and Directions

1. Range gradient

$$\frac{\partial d}{\partial x} = \frac{x - 20000}{d}$$

$$\frac{\partial d}{\partial y} = \frac{y - 15000}{d}$$

$$\frac{\partial d}{\partial z} = \frac{z - 25000}{d}$$

The gradient points from satellite to receiver. The formula requires nonzero range.

2. Bearing derivatives

Use local derivatives away from the angle wrap and away from coincident positions.

$$\frac{\partial \beta}{\partial x_1} = \frac{-(y_2 - y_1)}{(x_2 - x_1)^2 + (y_2 - y_1)^2}$$

$$\frac{\partial \beta}{\partial y_1} = \frac{(x_2 - x_1)}{(x_2 - x_1)^2 + (y_2 - y_1)^2}$$

3. Doppler convention

$$\frac{dd}{dt} = -\mathbf{e}^{\mathrm{T}}\mathbf{v}$$

$$f_D = \left(\frac{f_0}{c}\right)\mathbf{e}^{\mathrm{T}}\mathbf{v}$$

Here e points receiver to satellite. Motion toward the satellite gives positive received-frequency shift.

From Derivatives to Jacobians: One Variable

Single Variable Differentiation

For a function $f: \mathbb{R} \to \mathbb{R}$:

$$f'(x) = \lim_{h \to 0} \frac{f(x+h) - f(x)}{h}$$

Example: Distance from beacon

$$f(x) = \sqrt{(x - x_0)^2 + y_0^2}$$

$$f'(x) = \frac{x - x_0}{\sqrt{(x - x_0)^2 + y_0^2}}$$

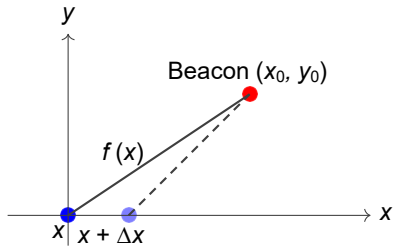
Linear Approximation:

$$f(x + \Delta x) \approx f(x) + f'(x) \cdot \Delta x$$

The derivative tells us:

- Rate of change
- Sensitivity to input changes
- Direction of steepest increase

Navigation Example: 1D Positioning



Physical Meaning:

- $f(x)$: measured distance
- $f'(x)$:how distance changes as receiver moves
- Sign indicates approaching/receding
- Magnitude shows sensitivity

From Derivatives to Jacobian Matrices - II: Multivariable Functions

Vector-Valued Functions
For $\mathbf{f}: \mathbb{R}^n \to \mathbb{R}^m$:

$$\mathbf{f}(\mathbf{x}) = \begin{bmatrix} f_1(x_1, x_2, ., .., \mathbf{x}_n) \\ f_2(x_1, x_2, ., .., \mathbf{x}_n) \\ \vdots \\ f_m(x_1, x_2, ., .., \mathbf{x}_n) \end{bmatrix}$$

The Jacobian Matrix:

$$\mathbf{J} = \frac{\partial \mathbf{f}}{\partial \mathbf{x}} = \begin{bmatrix} \frac{\partial f_1}{\partial x_1} & \frac{\partial f_1}{\partial x_2} & \cdots & \frac{\partial f_1}{\partial \mathbf{x}_n} \\ \frac{\partial f_2}{\partial x_1} & \frac{\partial f_2}{\partial x_2} & \cdots & \frac{\partial f_2}{\partial \mathbf{x}_n} \\ \vdots & \vdots & \ddots & \vdots \\ \frac{\partial f_m}{\partial x_1} & \frac{\partial f_m}{\partial x_2} & \cdots & \frac{\partial f_m}{\partial \mathbf{x}_n} \end{bmatrix}$$

Each row: gradient of one output function
Each column: sensitivity to one input variable

Example: 2D to 2D Transformation
Polar to Cartesian conversion:

$$\mathbf{f}(r, \theta) = \begin{bmatrix} \mathbf{x} \\ y \end{bmatrix} = \begin{bmatrix} r\cos\theta \\ r\sin\theta \end{bmatrix}$$

Jacobian matrix:

$$\mathbf{J} = \begin{bmatrix} \frac{\partial \mathbf{x}}{\partial r} & \frac{\partial \mathbf{x}}{\partial \theta} \\ \frac{\partial y}{\partial r} & \frac{\partial y}{\partial \theta} \end{bmatrix} = \begin{bmatrix} \cos\theta & -r\sin\theta \\ \sin\theta & r\cos\theta \end{bmatrix}$$

Linear Approximation:

$$\mathbf{f}(\mathbf{x} + \Delta\mathbf{x}) \approx \mathbf{f}(\mathbf{x}) + \mathbf{J}\Delta\mathbf{x}$$

Properties:

- $\det(\mathbf{J}) = r$ (area scaling factor)
- Singular when $r = 0$ (at origin)
- Shows how errors propagate

A Jacobian Predicts Small Changes

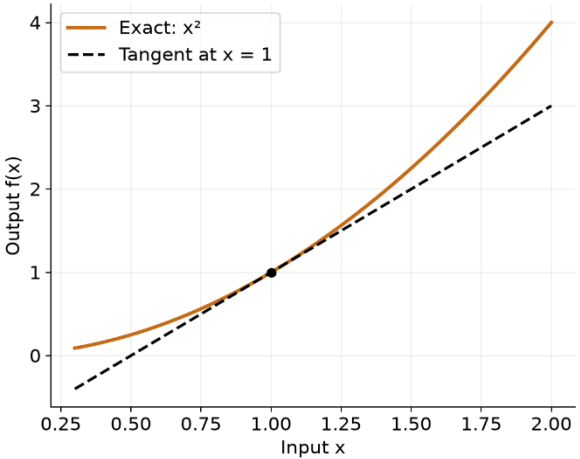
A tangent is a local model

$$f(x) = x^2$$

$$f(1 + \Delta x) \approx 1 + 2\Delta x$$

At a small step, the tangent closely follows the curve. A larger step exposes the neglected curvature.

A Jacobian applies this idea to several inputs and outputs. Recompute it when the linearization point changes.



Jacobian Example 1: Simple 2D Transformation

Function Definition:

$$\mathbf{f}(x, y) = \begin{bmatrix} u \\ v \end{bmatrix} = \begin{bmatrix} x^2 + 2y \\ 3x - y^2 \end{bmatrix}$$

Step 1: Find partial derivatives

$$\frac{\partial u}{\partial x} = 2x, \frac{\partial u}{\partial y} = 2$$

$$\frac{\partial v}{\partial x} = 3, \frac{\partial v}{\partial y} = -2y$$

Step 2: Form Jacobian matrix

$$\mathbf{J} = \begin{bmatrix} 2x & 2 \\ 3 & -2y \end{bmatrix}$$

Numerical Example:
At point $(x, y) = (1, 2)$:

$$\mathbf{f}(1,2) = \begin{bmatrix} 1^2 + 2(2) \\ 3(1) - 2^2 \end{bmatrix} = \begin{bmatrix} 5 \\ -1 \end{bmatrix}$$

Jacobian at (1, 2):

$$\mathbf{J}(1,2) = \begin{bmatrix} 2(1) & 2 \\ 3 & -2(2) \end{bmatrix} = \begin{bmatrix} 2 & 2 \\ 3 & -4 \end{bmatrix}$$

Linear approximation: Near (1, 2), for small $\Delta x, \Delta y$:

$$\mathbf{f}(1 + \Delta x, 2 + \Delta y) \approx \begin{bmatrix} 5 \\ -1 \end{bmatrix} + \begin{bmatrix} 2 & 2 \\ 3 & -4 \end{bmatrix} \begin{bmatrix} \Delta x \\ \Delta y \end{bmatrix}$$

Jacobian Example 2: Product and Ratio Functions

Function Definition:

$$\mathbf{f}(x,y) = \begin{bmatrix} u \\ v \end{bmatrix} = \begin{bmatrix} xy \\ x/y \end{bmatrix}$$

Partial derivatives:

$$\frac{\partial u}{\partial x} = y, \frac{\partial u}{\partial y} = x$$
$$\frac{\partial v}{\partial x} = \frac{1}{y}, \frac{\partial v}{\partial y} = -\frac{x}{y^2}$$

Jacobian matrix:

$$\mathbf{J} = \begin{bmatrix} y & x \\ 1/y & -x/y^2 \end{bmatrix}$$

Numerical Example:

At point $(x, y) = (6, 3)$:

$$\mathbf{f}(6,3) = \begin{bmatrix} 6 \times 3 \\ 6/3 \end{bmatrix} = \begin{bmatrix} 18 \\ 2 \end{bmatrix}$$

Jacobian at (6, 3):

$$\mathbf{J}(6,3) = \begin{bmatrix} 3 & 6 \\ 1 & -\frac{2}{3} \\ 3 & \end{bmatrix}$$

Check with small perturbation:

$$\mathbf{f}(6.1,3.1) \approx \begin{bmatrix} 18.91 \\ 1.9677 \end{bmatrix}$$

Linear approximation gives:

$$\begin{bmatrix} 18 \\ 2 \end{bmatrix} + \begin{bmatrix} 3 & 6 \\ 1 & -\frac{2}{3} \\ 3 & \end{bmatrix} \begin{bmatrix} 0.1 \\ 0.1 \end{bmatrix} \approx \begin{bmatrix} 18.90 \\ 1.9667 \end{bmatrix}$$

Jacobian Example: Exact Change Versus Approximation

Use the product-and-ratio example

$$\mathbf{f}(x, y) = \begin{bmatrix} xy \\ \frac{x}{y} \end{bmatrix}$$

$$\mathbf{J}(6,3) = \begin{bmatrix} 3 & 6 \\ \frac{1}{3} & -\frac{2}{3} \end{bmatrix}$$

$$\Delta\mathbf{x} = \begin{bmatrix} 0.1 \\ 0.1 \end{bmatrix}, \mathbf{J}\Delta\mathbf{x} = \begin{bmatrix} 0.9 \\ -0.0333 \end{bmatrix}$$

Keep the calculation visible

$$6.1 \times 3.1 = 18.91$$

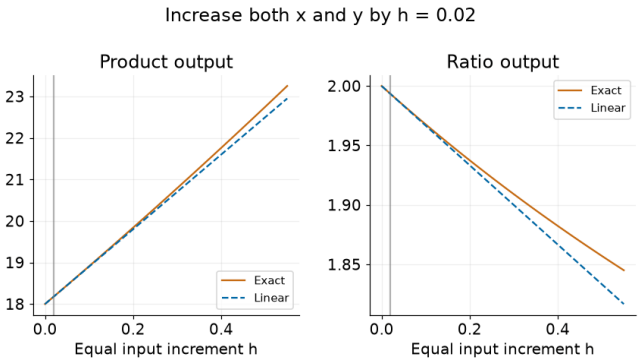
$$\frac{6.1}{3.1} \approx 1.9677$$

Linear prediction: (18.90, 1.9667).
Exact evaluation: (18.91, 1.9677).

The product misses $\Delta x \Delta y = 0.01$. Smaller perturbations reduce the neglected nonlinear terms.

Jacobian Animation: Why Step Size Matters

Increase both inputs from (6, 3) by the same amount h. Compare each exact output with its linear prediction.



For h = 0.10, the product is 18.91; its linear prediction is 18.90.

For h = 0.50, the product is 22.75; its linear prediction is 22.50.

The neglected product term is h squared. The local model improves as h decreases.

$$(6 + h)(3 + h) = 18 + 9h + h^2$$

Jacobian Example 3: Three Variables to Two Outputs

Function Definition:

$$\mathbf{f}(x,y,z) = \begin{bmatrix} u \\ v \end{bmatrix} = \begin{bmatrix} x+y+z \\ x^2+y^2+z^2 \end{bmatrix}$$

Partial derivatives:

$$\frac{\partial u}{\partial x} = 1, \frac{\partial u}{\partial y} = 1, \frac{\partial u}{\partial z} = 1$$

$$\frac{\partial v}{\partial x} = 2x, \frac{\partial v}{\partial y} = 2y, \frac{\partial v}{\partial z} = 2z$$

Jacobian matrix (2×3):

$$\mathbf{J} = \begin{bmatrix} 1 & 1 & 1 \\ 2x & 2y & 2z \end{bmatrix}$$

Note: Non-square Jacobian!

Numerical Example:
At point (*x*, *y*, *z*) = (1, 2, 3):

$$\mathbf{f}(1,2,3) = \begin{bmatrix} 1+2+3 \\ 1^2+2^2+3^2 \end{bmatrix} = \begin{bmatrix} 6 \\ 14 \end{bmatrix}$$

Jacobian at (1, 2, 3):

$$\mathbf{J}(1,2,3) = \begin{bmatrix} 1 & 1 & 1 \\ 2 & 4 & 6 \end{bmatrix}$$

Interpretation:

- First row: equal sensitivity to all inputs
- Second row: *z* has strongest effect on *v*
- Can't invert (not square) For $\Delta x = [0.1, 0.1, 0.1]^{\mathrm{T}}$:

$$\Delta \mathbf{f} \approx \mathbf{J}\Delta x = \begin{bmatrix} 0.3 \\ 1.2 \end{bmatrix}$$

Jacobian Calculation: Summary and Practice

Steps to Calculate Jacobian:

1. Identify input and output dimensions
2. Calculate all partial derivatives
3. Arrange in matrix form:

$$j_{\mathrm{ij}} = \frac{\partial f_i}{\partial x_j}$$

4. Evaluate at specific point if needed

Quick Practice: Find the Jacobian of:

$$\mathbf{f}(x, y) = \begin{bmatrix} 2x + y \\ x - 3y \\ xy \end{bmatrix}$$

Solution:

$$\mathbf{J} = \begin{bmatrix} \frac{\partial f_1}{\partial x} & \frac{\partial f_1}{\partial y} \\ \frac{\partial f_2}{\partial x} & \frac{\partial f_2}{\partial y} \\ \frac{\partial f_3}{\partial x} & \frac{\partial f_3}{\partial y} \end{bmatrix} = \begin{bmatrix} 2 & 1 \\ 1 & -3 \\ y & x \end{bmatrix}$$

At point $(x, y) = (4, 2)$:

$$\mathbf{J}(4,2) = \begin{bmatrix} 2 & 1 \\ 1 & -3 \\ 2 & 4 \end{bmatrix}$$

Key Points:

- Size: 3×2 (3 outputs, 2 inputs)
- Some entries constant, some variable
- Each row = gradient of one output

Range Jacobians in Navigation

Geometric ranges with known clock bias

$$\rho_i = \sqrt{(x - x_i)^2 + (y - y_i)^2 + (z - z_i)^2}$$

$$h_{ij} = \frac{\partial \rho_i}{\partial x_j}$$

Each row has three entries: the components of the unit vector from satellite to receiver.

$$\Delta\boldsymbol{\rho} \approx \mathbf{H}\Delta\mathbf{x}$$

All ranges must be nonzero.

Two-beacon example

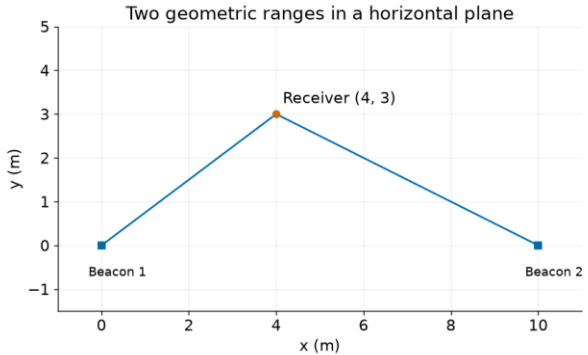
Beacons are at (0, 0) and (10, 0).

$$\mathbf{H} = \begin{bmatrix} \frac{x}{\rho_1} & \frac{y}{\rho_1} \\ \frac{x-10}{\rho_2} & \frac{y}{\rho_2} \end{bmatrix}$$

Diverse row directions improve local observability. Collinear rows leave a direction unresolved.

Range Jacobian Example: Geometry and Input Data

Two fixed beacons measure distance to a receiver. Coordinates and ranges are in metres.



Receiver position: (4, 3). Beacon positions: (0, 0) and (10, 0).

Range 1 is 5.00 m. Range 2 is the square root of 45, about 6.71 m.

Predict how both ranges change when the receiver moves 0.10 m right and 0.05 m down.

$$\rho_1 = \sqrt{4^2 + 3^2} = 5, \qquad \rho_2 = \sqrt{6^2 + 3^2} = \sqrt{45}$$

Range Jacobian Example: Predict and Verify

Step 1: Form the two Jacobian rows

$$\mathbf{H} = \begin{bmatrix} \frac{4}{5} & \frac{3}{5} \\ -\frac{6}{\sqrt{45}} & \frac{3}{\sqrt{45}} \end{bmatrix}$$

$$\mathbf{H} \approx \begin{bmatrix} 0.8000 & 0.6000 \\ -0.8944 & 0.4472 \end{bmatrix}$$

Each row is the unit direction from the corresponding beacon to the receiver.

Step 2: Multiply by the displacement

$$\Delta\mathbf{p} = \begin{bmatrix} 0.10 \\ -0.05 \end{bmatrix}$$

$$\mathbf{H}\Delta\mathbf{p} \approx \begin{bmatrix} 0.0500 \\ -0.1118 \end{bmatrix}$$

Step 3: Recompute at (4.10, 2.95).
Exact range changes: about
+0.0510 m and -0.1118 m.

Range 2 is exact here: the motion is radial.

GNSS Pseudorange: Add the Receiver Clock

A practical positioning state has four unknowns

$$\mathbf{x} = [x, y, z, b]^{\mathrm{T}}, \qquad b = c\delta t$$

After applying satellite-clock and propagation corrections, model each pseudorange with a geometric range plus receiver clock bias.

$$r_i = \rho_i + b + \varepsilon_i$$

$$\mathbf{h}_i = \left[-e_{ix}, -e_{iy}, -e_{iz}, 1\right]$$

The unit vector e points receiver to satellite. At least four independent measurement rows are needed for a local unconstrained solution of the four unknowns.

Summary and Key Takeaways



Advanced matrix properties
Eigenvalues describe invariant directions and scaling.
Positive definiteness describes a positive quadratic form.
Condition numbers describe numerical sensitivity.
A suitable decomposition exploits the matrix structure.

Matrix Decompositions

- Different decompositions for different problems
- Selection based on problem structure
- Navigation applications have specific requirements
- Computational efficiency vs. numerical stability

Singular Value Decomposition

- Universal decomposition for any matrix
- Reveals key matrix properties (rank, condition)
- Provides stable solutions to linear systems
- Handles rank deficiency naturally
- Offers regularization methods for ill-conditioned problems

Navigation Applications

- GNSS position estimation
- Sensor calibration
- State estimation
- Data compression
- System analysis
- Filtering algorithms

Matrix Calculation Tools and Solvers

Software Libraries
General-purpose mathematical:

- **MATLAB/Octave**
  - Built-in matrix operations
  - eig, svd, inv, pinv
  - Optimized for matrix manipulations
- **Python Ecosystem**
  - NumPy: np.linalg module
  - SciPy: Advanced decompositions
  - SymPy: Symbolic matrix calculations
- **Specialized Libraries**
  - LAPACK/BLAS: Industry standard
  - Eigen (C++): Template library
  - Armadillo, Blaze, Eigen3

Navigation-Specific Tools
Navigation libraries:

- **RTKLIB**: GNSS positioning algorithms
- **GeographicLib**: Geodetic transformations
- **GPSTK**: GPS toolkit with matrix utilities
- **ROS tf2**: Transformation framework

Implementation Considerations:

- **Embedded systems**:
  - Fixed-point vs. floating-point
  - Memory constraints
  - Real-time requirements
- **Numerical stability**:
  - Avoid direct inversion when possible
  - Use decompositions (LU, QR, SVD)
  - Consider condition number
  - Regularization techniques

Matrix Computation Tutorial - Part 1: Introduction



Tutorial Overview

Objective: Compare computational accuracy and efficiency of matrix operations between C++/Eigen and Python/NumPy

Learning Outcomes:

- Understand performance differences between C++ and Python for matrix operations
- Identify accuracy trade-offs in different numerical implementations
- Learn to properly benchmark numerical algorithms
- Apply findings to navigation algorithm implementation decisions

Setup Instructions:

1. Download starter code from course website
2. Install required libraries:
   - C++: Eigen, Chrono
   - Python: NumPy, SciPy, time, memoryprofiler
3. Follow README instructions to compile C++ code
4. Use provided scripts to run comparison tests

Matrix Computation Tutorial - Part 2: Operations to Implement



Operations to Implement:

- Matrix multiplication
  - Standard matrix product
  - Element-wise multiplication
- Matrix inversion
  - Direct inversion
  - Using decompositions
- Eigenvalue decomposition
  - Full decomposition
  - Only eigenvalues
- Singular Value Decomposition
  - Full vs. reduced SVD
- Least squares solution

Testing Parameters:

Matrix dimensions:

- Small: 10×10
- Medium: 100×100
- Large: 1000×1000

Data types:

- Single precision (float)
- Double precision (double)

Matrix types:

- Random matrices
- Nearly singular matrices
- Structured matrices (e.g., diagonal dominant)
- Orthogonal matrices

Number of iterations:

- 10, 100, 1000 (operation dependent)

Matrix Computation Tutorial - Part 3: Methodology



Metrics to Measure:

- **Execution time**
  - Wall clock time
  - CPU time
- **Memory usage**
  - Peak memory
  - Memory growth
- **Numerical accuracy**
  - Frobenius norm error
  - Maximum element-wise error
  - Relative error
- **Scaling behavior**
  - Time vs. matrix size
  - Memory vs. matrix size

Accuracy Comparison:

- Use higher precision library for reference solutions
- Test with well-conditioned and ill conditioned matrices
- Verify correctness of operations ($\mathbf{A} \times \mathbf{A}^{-1} = \mathbf{I}$)

Performance analysis
Use the same input arrays and numeric precision.
Match requested outputs and BLAS thread counts.
Warm up, repeat, and report median timings.
Record hardware, library versions and compiler flags.
NumPy and Eigen may use different SVD algorithms.

Matrix Computation Tutorial - Part 4: Code Examples



Sample Code Snippets

C++/Eigen:

```cpp
#include <Eigen/Dense>
#include <chrono>
#include <cmath>
#include <iostream>
int main() {
  Eigen::MatrixXd A(100,100);
  for(int i=0;i<100;++i)
    for(int j=0;j<100;++j)
      A(i,j)=std::sin((i+1)*(j+1))
             +0.1*(i==j);
  auto t=std::chrono::steady_clock::now();
  Eigen::JacobiSVD<Eigen::MatrixXd> svd(A);
  auto dt=std::chrono::steady_clock::now()-t;
  std::cout << std::chrono::duration<double>
    (dt).count() << " seconds\n";
}
```

Python/NumPy:

```python
import numpy as np
from time import perf_counter

i = np.arange(1, 101)
A = np.sin(np.outer(i, i))
A += 0.1 * np.eye(100)

t = perf_counter()
s = np.linalg.svd(A, compute_uv=False)
elapsed = perf_counter() - t
print(f"{elapsed:.6f} seconds")

# Both examples compute singular values.
# Algorithms can differ across libraries.
# Warm up and repeat for benchmarking.
```

Laboratory Extensions: Mathematics in Use

The same matrix tools connect sensor geometry, data compression and robot motion.

SVD and data size

Approximate a sampled field with fewer singular modes. Compare reconstruction error with stored coefficients.

Geometry and uncertainty

Keep range noise fixed and move the beacons. Measure how weak geometry amplifies position error.

Jacobian and control

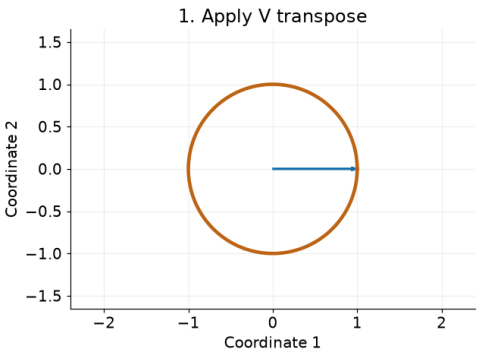
Convert joint motion into tip motion. Observe the direction lost at a singular configuration.

Suggested classroom use

Predict the animation, watch it, then reproduce one result in Python or C++.

SVD Animation: How a Matrix Acts

A 2D example makes the order of the factors visible. The dashed circle is the original input set.



Observe the blue vector and the whole circle together.

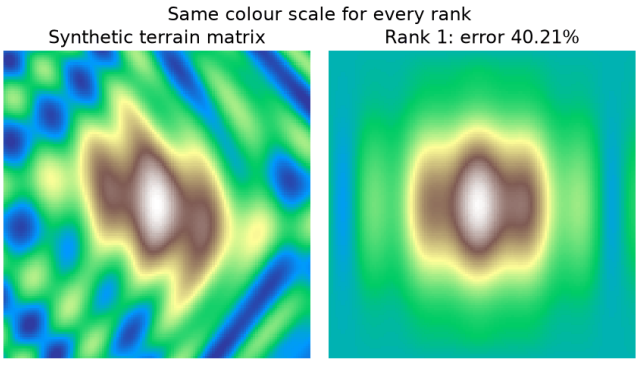
Here the two singular values are 2 and 0.70. They set the final ellipse semi-axes.

Predict: which factor changes lengths? General orthogonal factors can also reflect.

$$\mathbf{A} = \mathbf{U}\mathbf{\Sigma}\mathbf{V}^{\mathrm{T}}$$

Application: SVD Compression of a Sampled Field

A synthetic terrain field is stored as a 128 by 128 matrix. The colour scale stays fixed as rank increases.



$$\mathbf{A}_k = \mathbf{U}_k \mathbf{\Sigma}_k \mathbf{V}_k^{\mathrm{T}}$$

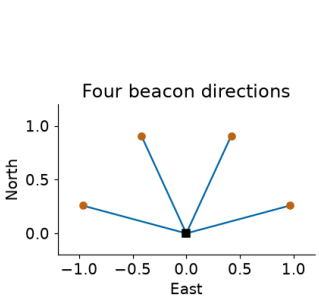
A small rank retains dominant spatial patterns but loses finer detail.

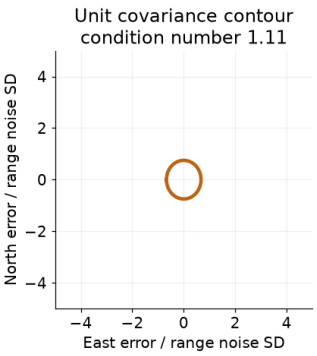
Try ranks 1, 2, 4, 8 and 16. Compare error with k(128+128+1) stored scalars.

This counts factor coefficients, not an encoded file size. High ranks may use more storage.

Application: Range Geometry and Position Error

A 2D beacon model with known clock bias isolates geometry. Four independent range errors have equal variance.





The receiver stays fixed. Beacon directions become clustered, then spread out.

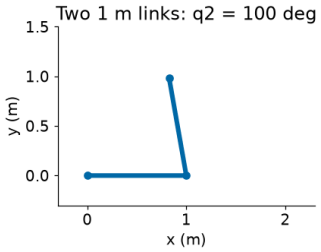
The error contour stretches along the poorly observed direction. Its scale stays fixed.

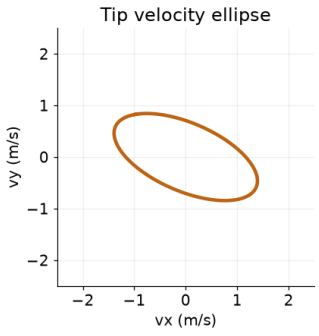
For GNSS, also estimate clock bias. This 2D illustration is not a full PDOP calculation.

$$\mathbf{P} = \sigma^2 \left(\mathbf{H}^{\mathrm{T}}\mathbf{H}\right)^{-1}$$

Application: Robot Jacobians and Singularities

Two planar links each have length 1 m. The first joint angle is zero while the second approaches zero.





The orange curve maps unit joint-speed vectors into possible tip velocities.

As the arm straightens, one singular value approaches zero.

At the straight configuration, the tip has no instantaneous horizontal velocity.

$$\Delta \mathbf{p} \approx \mathbf{J} \Delta \mathbf{q}$$

Laboratory Tasks: SVD and Geometry



Use the same data and units when comparing implementations. Save the numerical output as well as the plots.

Experiment A: matrix approximation

1. Load a grayscale image or sampled field.
2. Compute its SVD once.
3. Reconstruct at several ranks.
4. Plot relative Frobenius error against rank.

Experiment B: range geometry

1. Build H from four 2D unit directions.
2. Compare spread and clustered directions.
3. Compute singular values and covariance.
4. Draw the covariance contour.

Report: explain one accuracy-versus-storage tradeoff and one weakly observed position direction. State the noise and clock assumptions.

Laboratory Task: Checking a Robot Jacobian

For unit links, use joint angles in radians. Recompute the Jacobian at each configuration.

Forward model

$$x = \cos(q_1) + \cos(q_1 + q_2)$$
$$y = \sin(q_1) + \sin(q_1 + q_2)$$

Numerical check

Perturb one angle at a time by a small amount. Divide the position change by that perturbation and compare with the Jacobian column.

Singularity experiment

Compare q2 = 90°, 30°, 5° and 0°. Plot the smallest singular value. Explain the lost direction at 0°.

Expected check

$$\det(\mathbf{J}) = \sin(q_2)$$

Video Resources for the Laboratory

Click an underlined title to open the external video. Internet access is required. The ten embedded GIFs work offline.

3Blue1Brown: Linear transformations and matrices

Watch for how the images of basis vectors determine the matrix.

Steve Brunton: SVD Image Compression [Python]

Connect the rank parameter to reconstruction quality and storage.

Modern Robotics: Chapter 5.3, Singularities

Relate a loss of Jacobian rank to a lost motion direction.

Questions?

Dr. Weisong Wen
Assistant Professor

Trustworthy AI and Autonomous Systems Laboratory (TAS Lab)
Department of Aeronautical and Aviation Engineering
The Hong Kong Polytechnic University

Email: *welson.wen@polyu.edu.hk*
Office: R820 **Phone**: 3400 8234